

1. Область видимости и время жизни переменных. Признаки базы данных. Определение базы данных.

Основные области видимости:

- Глобальная (объявлена вне всех функций, доступ к ней осуществляется из любого места проги. Если время жизни совпадает с временем жизни программы, то переменная тоже глобальная)
- Локальная (объявлена внутри блока, принадлежит локальной области, не видна за пределами блока, в котором объявлена. Расположена в стеке и ее время жизни совпадает с временем жизни функции).

Одной из причин появления БД является потребность в увеличении времени жизни переменных:

- сеансовые: блока кода, функции, модуля, программы;
- между сеансами: между запусками программы.

Признаки базы данных:

- хранятся и обрабатываются в вычислительной системе – требует формального языка,
- имеют логическую структуру - требуют типизации,
- описываются формально описанными схемой или метаданными.

• База данных (БД) — совокупность данных, хранимых в соответствии со схемой данных, манипулирование которыми выполняют в соответствии с правилами средств моделирования данных (ГОСТ)

• База данных (БД) — организованная в соответствии с определенными правилами и поддерживаемая в памяти компьютера совокупность данных, характеризующая актуальное состояние некоторой предметной области и используемая для удовлетворения информационных потребностей пользователей. (Когаловский М.Р.)

• База данных (БД) — некоторый набор перманентных (постоянно хранимых) данных, используемых прикладными программными системами какого-либо предприятия (Деят К.Дж.)

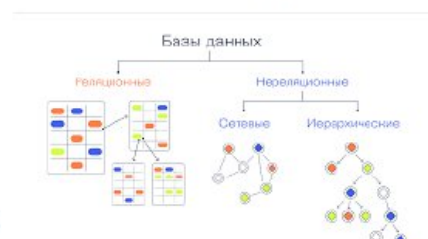


2. Понятие модели данных. Виды модели данных.

Для формализации обработки в вычислительной системе требуется зафиксировать перечень операций и определить множество, на котором эти операции выполняются, то есть задать **модель данных**.

Модели данных:

- Иерархическая – основная структура – дерево.
- Сетевая – основная структура – граф.
- Реляционная – основная структура – отношение.



Не являются базами данных файловые архивы, интернет-порталы, электронные таблицы

3. Иерархическая модель данных. Основной объект, методы управления данными.

Основной элемент – n-арное дерево

Схема данных иерархической БД



Операции в иерархической БД:

- найти экземпляр типа,
- перейти к следующему экземпляру,
- перейти вниз по иерархии дерева,
- перейти к следующему элементу на уровне,
- вставить новую запись,
- удалить запись.

Первая реализация IMS IBM 1968 год.

Пример заполнения данными в иерархической БД



Достоинства иерархической базы данных:

- высокое быстродействие;
- Простота параллельного доступа.

Недостатки иерархической базы данных:

- трудность в восприятии обычным пользователем;
- сложность управления и реализации СУБД;
- трудность в применении.
- аномалии данных.

4. Сетевая модель данных. Основной объект, методы управления данными.

Основной элемент – ациклический направленный граф (сеть)

Расширение модели: в сетевой модели у потомка может быть любое число предков

Схема данных сетевой БД



Описание схемы:

Тип записей: Отдел, Руководитель, Служащие

Тип связи (Предок - Потомок):

- отдел-служащие,
- отдел-руководитель,
- руководитель-служащие.

Первая реализация IDMS IBM 1972 год. Пример dbVista 1990.

Операции в иерархической БД:

- найти конкретную запись в наборе,
- перейти от предка к первому потомку,
- перейти к следующему потомку,
- перейти от потомка к предку,
- вставить новую запись,
- удалить запись,
- модифицировать запись,
- включить в связь,
- исключить из связи,
- переставить в другую связь.

Достоинства сетевой модели данных:

- высокое быстродействие;
- стандартизация на основе CODASYL;
- использование теоретико-графовых моделей

Недостатки сетевой модели данных:

- жесткость задаваемых структур,
- сложная структура управления памятью.



Чарльз
Charles 1

5. Реляционная модель данных. Основной объект, методы управления данными.

Достоинства реляционной модели данных:

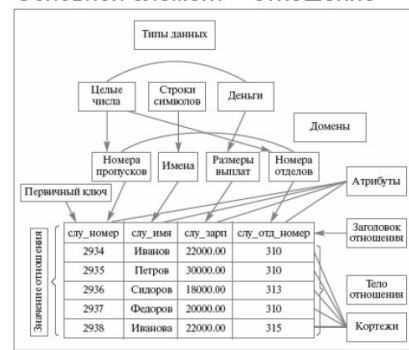
- теоретическая основа. Формально определяет базовые понятия модели;
- простота манипуляции данными с точки зрения конечного пользователя.

Недостатки реляционной модели данных:

- низкое быстродействие по сравнению с сетевыми и иерархическими СУБД;
- Требования учета модели на концептуальном уровне.

Термин РМД	Англ. термин	Неформальный термин
Отношение	Relation	Таблица
Кортеж	Tuple	Запись таблицы
Атрибут	Attribute	Столбец таблицы
Домен	Domain	Тип данных у значений в столбце таблицы
Первичный ключ	Primary key	Поле - уникальный идентификатор записи

Основной элемент – отношение



Методы модификации данных: добавить, удалить, модифицировать запись

Методы обработки данных по Кодду: выборка, проекция, декартово произведение, объединение, вычитание

Оставшиеся операции: соединение, пересечение, деление

6. Домен. Отношение. Схема данных.

Домен – совокупность всех значений атрибута.

Отношение – совокупность всех кортежей и заголовка:

- заголовок отношения содержит имена атрибутов (A) и домены (T),
- тело отношения содержит кортежи.

Число кортежей – **кардинальное число**.

Число атрибутов – **степень** отношения.

Заголовок $H = \{ \langle A_1; T_1 \rangle; \langle A_2; T_2 \rangle; \dots; \langle A_n; T_n \rangle \}$.

Тело B_{R_i} . $B_{R_i} = \{ \langle A_{1i}, T_{1i}, v_{1i} \rangle; \langle A_{2i}, T_{2i}, v_{2i} \rangle, \dots, \langle A_{nii}, T_{nii}, v_{nii} \rangle \}$

Заголовок	$\langle A_1, T_1 \rangle$	$\langle A_2, T_2 \rangle$	...	$\langle A_n, T_n \rangle$
Тело	$\langle A_1, T_1, v_{11} \rangle$	$\langle A_2, T_2, v_{12} \rangle$	...	$\langle A_n, T_n, v_{1n} \rangle$
	$\langle A_1, T_1, v_{21} \rangle$	$\langle A_2, T_2, v_{22} \rangle$		$\langle A_n, T_n, v_{2n} \rangle$

$\langle \text{Имя, string} \rangle$	$\langle \text{дата рождения, date} \rangle$
$\langle \text{Имя, string, Вася} \rangle$	$\langle \text{дата рождения, date, 6/10/1980} \rangle$
$\langle \text{Имя, string, Петя} \rangle$	$\langle \text{дата рождения, date, 1/19/1991} \rangle$

Схема данных: $H_i = \{ \langle A_{1i}, T_{1i} \rangle; \langle A_{2i}, T_{2i} \rangle; \dots \langle A_{ni}, T_{ni} \rangle \}$.

7. Понятие ключа. Первичный ключ. Внешний ключ. Простой ключ, составной ключ. Естественные и искусственные ключи.

Ключ – атрибут (группа атрибутов), которые позволяют классифицировать кортеж отношения (запись таблицы).

Суперключ – атрибут или множество атрибутов, которое единственным образом идентифицирует кортеж данного отношения. (Может содержать «лишние» атрибуты).

Потенциальный ключ – суперключ, который не содержит подмножества, также являющегося суперключом данного отношения. Потенциальный ключ K обладает двумя свойствами:

- Уникальность (в рамках кортежей данного отношения);
- Неприводимость: никакое допустимое подмножество ключа K не обладает свойством уникальности.

Первичный ключ (ПК) – потенциальный ключ, выбранный для однозначной идентификации кортежей отношения. Значения всех атрибутов первичного ключа являются обязательными. Для каждого отношения может быть определен только один первичный ключ.

Вторичный ключ – любой другой ключ, кроме первичного. Может быть необязательным.

Внешний ключ – атрибут или множество атрибутов отношения, которое соответствует потенциальному ключу некоторого (возможно, того же самого) отношения. Обычно служит для организации связей между таблицами.

- Суперключ – ключ, который гарантирует уникальность (может быть избыточным)
- Потенциальный ключ – минимальный суперключ, из которого нельзя удалить ни один атрибут без потери уникальности

Если первичный ключ состоит из единственного атрибута, то его называют простым.

Если первичный ключ состоит из более чем 2 атрибутов, то это составной ключ

Естественные ключи – основаны на естественных свойствах данных, таких как номер страхового полиса и тд

Искусственные ключи создаются специально для идентификации записей

8. Обзор теоретико-множественных операции над отношениями. Объединение. Пересечение. Вычитание. Декартово произведение.

- Теоретико-множественные:
- объединение (union)
 - пересечение (intersect)
 - вычитание (minus)
 - взятие расширенного декартова произведения (times)
- Дополнительный набор операций:
- переименование атрибутов (rename)
 - выборка (where)
 - проекция (project)
 - соединение (join)
 - деление (divide by)
 - присваивание (значения)

Унарные операции: выборка, проекция, переименование.

Бинарные операции: объединение, пересечение, вычитание, декартово произведение, соединение, деление, присваивание.

Проблема целостности данных:

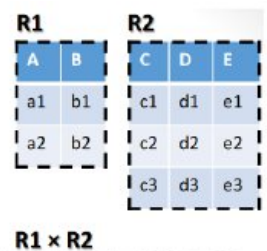
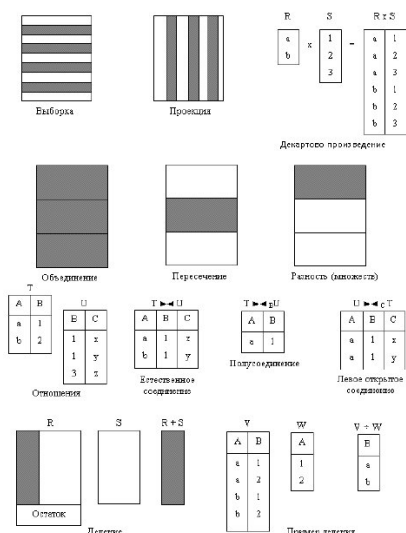
- Ограничения целостности сущности (единственность атрибута)
- Ограничения ссылочной целостности (поддержание корректных ссылок предок - потомок)

Не совсем алгебра – некоторые операции являются частичными

➤ Декартово произведение (times)

Это бинарная операция над односхемными отношениями, соответствующая определению декартова произведения для РМД: в результате получается отношение, схема которого включает все атрибуты исходных отношений.

Декартовым произведением отношений $R_{A_1, \dots, A_n}$ и $S_{B_1, \dots, B_k}$ называют отношение $T_{A_1, \dots, A_n, B_1, \dots, B_k}$, содержащее все комбинации кортежей исходных отношений.



Объединение (union)

Объединение – бинарная операция над односхемными отношениями.

Объединением отношений $R_{A_1, \dots, A_n}$ и $S_{A_1, \dots, A_n}$ называют отношение

$T_{A_1, \dots, A_n}$, при этом кортеж t_i входит в T , если он принадлежит R или S :

$$t_i \in T: t_i \in R \vee t_i \in S.$$

То есть объединение включает в себя все кортежи обоих исходных отношений без повторов.

Вычитание (minus)

Разность – бинарная операция над односхемными отношениями.

Разностью $T = R - S$ отношений $R_{A_1, \dots, A_n}$ и $S_{A_1, \dots, A_n}$ называется множество кортежей R , не входящих в S :

$$t_i \in T: t_i \in R \wedge \text{not}(t_i \in S).$$

Пересечение (intersect)

Пересечение – бинарная операция над односхемными отношениями.

Пересечением $T = R \cap S$ отношений $R_{A_1, \dots, A_n}$ и $S_{A_1, \dots, A_n}$ называют отношение $T_{A_1, \dots, A_n}$, при этом кортеж t_i входит в T , если он принадлежит обоим исходным отношениям

$$t_i \in T: t_i \in R \wedge t_i \in S.$$

Это можно выразить через разность:

$$R \cap S = R - (R - S).$$

9. Обзор операций переименования, выборки, проекции соединения и деления.

Операция	Обозначение греч. алф.	Обозначение лат. алф.
Выборка (ограничение)	$\sigma_{condition}(R)$	R WHERE condition
Проекция	$\pi_{Attr1, \dots, AttrN}(R)$	R[Attr1, ..., AttrN]
Естественное соединение	$R1 \bowtie R2$	R1 JOIN R2
Θ-соединение	$R1 \bowtie_{R1.Attr1 \Theta R2.Attr2} R2$	-
Деление	$R1 \div R2$	R1 DIVIDE R2
Группировка	$\gamma_{Attr1, \dots, AttrN}(R)$	
Сортировка	$\tau_{Attr1, \dots, AttrN}(R)$	
Удаление дубликатов	$\delta_{Attr1, \dots, AttrN}(R)$	

➤ Проекция (project)

Проекцией π отношения $R_{A_1, A_2, \dots, A_n}$ на атрибуты $A_i, A_j, \dots, A_k$ называют отношение $R_{A_i, A_j, \dots, A_k}$, где $\{A_i, A_j, \dots, A_k\} \subset \{A_1, A_2, \dots, A_n\}$.

Это унарная операция, служащая для выбора подмножества атрибутов из отношения R.

Она уменьшает степень отношения и может уменьшить мощность отношения за счёт исключения одинаковых кортежей.



➤ Выборка (where)

Селекцией σ отношения $R_{A_1, A_2, \dots, A_n}$ по условию F называют отношение $R_{A_1, A_2, \dots, A_n}$, содержащее кортежи исходного отношения, в которых значения атрибутов удовлетворяют условию F .

Это унарная операция, результатом которой является подмножество кортежей исходного отношения, соответствующих условиям, которые накладываются на значения определённых атрибутов.

➤ Переименование (rename)

Переименование атрибутов β отношения $R_{A_1, A_2, \dots, A_n}$ по условию $A_1=F$ называют отношение $R_{F, A_2, \dots, A_n}$, содержащее все кортежи исходного выражения кортежи исходного отношения, с изменённым именем атрибута A_1 на F .

Это унарная операция, результатом которой является исходное отношение в заголовке нового установлено новое имя атрибута.

Деление (division)

Деление – это бинарная операция над двумя разносхемными отношениями.

Пусть отношение R содержит атрибуты $\{r_1, r_2, \dots, r_k, r_{k+1}, \dots, r_n\}$, а отношение S – атрибуты $\{r_{k+1}, \dots, r_n\}$. Тогда результирующее отношение T содержит атрибуты $\{r_1, r_2, \dots, r_k\}$. Кортеж t_i входит в T, если его декартово произведение с отношением S входит в R:

$$t_i \in T : t_i \times S \subset R.$$

Деление может быть выражено так:

$$R / S = \pi_{r_1, \dots, r_k}(R) - \pi_{r_1, \dots, r_k}((\pi_{r_1, \dots, r_k}(R) \times S) - R).$$

Соединение (join)

Соединение – бинарная операция над разносхемными отношениями. Эта операция определяет подмножество декартова произведения двух разносхемных отношений.

Кортеж декартова произведения входит в результирующее отношение, если выполняется условие соединения F, которое задаёт соотношение значений атрибутов разных таблиц. Соединение может быть выражено так:

$$R \bowtie S = \sigma_F(R \times S)$$

R1

A	B
a1	b1
a2	b2

Если условием является равенство значений двух атрибутов исходных отношений, такая операция называется **эквисоединением**.

Естественным называется эквисоединение по одинаковым атрибутам исходных отношений.

10. Операция проекции. Пример операции.

➤ Проекция (project)

Проекцией π отношения $R_{A_1, A_2, \dots, A_n}$ на атрибуты $A_i, A_j, \dots, A_k$ называют отношение $R_{A_i, A_j, \dots, A_k}$, где $\{A_i, A_j, \dots, A_k\} \subset \{A_1, A_2, \dots, A_n\}$.

Это унарная операция, служащая для выбора подмножества атрибутов из отношения R.

Она уменьшает степень отношения и может уменьшить мощность отношения за счёт исключения одинаковых кортежей.



Отношение R			Проекция $\pi_{A,C}(R)$	
A	B	C	A	C
a	b	c	a	c
c	a	d	c	d
c	b	d		

SELECT - выбор данных из таблиц в память.

- Однотабличные запросы.
- Многотабличные запросы соединением данных.
- Агрегатные функции.
- Группировка данных.
- Сортировка данных.
- Вложенные запросы.

Общий синтаксис:

```
SELECT [{ ALL | DISTINCT }] { список_вывода | * }
FROM имя_таблицы1 [ алиас1 ] [, имя_таблицы2 [ алиас2 ],...]
[ WHERE условие_отбора_записей ]
[ GROUP BY { имя_поля | выражение },... ]
[ HAVING условие_отбора_групп ]
[ ORDER BY имя_поля1 | целое [ ASC | DESC ]
[, имя_поля2 | целое [ ASC | DESC ],...]];
```

Общий синтаксис списка вывода:

```
{[all | distinct]} { * | выражение1 [алиас1] [, выражение2 [алиас2] ,...]}
```

Общий синтаксис списка вывода:

ALL - выбор всех записей

DISTINCT - исключение повторяющихся значений

* - выбор всех атрибутов - крайне нежелательно!

Пример: вывести ФИО посетителей, которые не имеют медицинской справки

Select (v.surname, v.name, v.patronymic) from Visitor v where medical_certificate = false;

11. Операция выборки. Пример операции.

➤ Выборка (where)

Селекцией σ отношения $R_{A_1, A_2, \dots, A_n}$ по условию F называют отношение $R_{A_1, A_2, \dots, A_n}$, содержащее кортежи исходного отношения, в которых значения атрибутов удовлетворяют условию F .

Это унарная операция, результатом которой является подмножество кортежей исходного отношения, соответствующих условиям, которые накладываются на значения определённых атрибутов.

WHERE - условие выборки

Содержит пары «имя атрибута» «значение атрибута»

Пары соединяются знаками (>, <, <=, >=, !=)

Конструируется по правилам логического выражения: and, or, !, ().

Используются специальные синтаксические конструкции:

- IN,
- BETWEEN,
- LIKE
- IS [NOT] NULL.

WHERE - выполнение выборки.

WHERE - примеры

ID = 5

Date > '2002-12-12' Cost <= 100
 (Date > '2002-12-12') and (Cost <= 100)
 (Date > '2002-12-12') or (Cost <= 100)
 ID IN (3,5,100)
 ID IN (Select)
 Date BETWEEN '2002-12-12' AND '2002-12-18'.
 Like 'a%'
 Like '_a'

12. Операция переименования. Пример операции.

➤ Переименование (rename)

Переименование атрибутов β отношения $R_{A_1, A_2, \dots, A_n}$ по условию $A_1 = F$ называют отношение $R_{F, A_2, \dots, A_n}$, содержащее все кортежи исходного выражения кортежи исходного отношения, с изменённым именем атрибута A_1 на F .

Это унарная операция, результатом которой является исходное отношение в заголовке нового установлено новое имя атрибута.

Alias - псевдоним нужен для формирования нового имени атрибута или отношения.

Пример: Select surname as 'фамилия'

13. Операция декартового произведения. Пример операции.

➤ Декартово произведение (times)

Это бинарная операция над разносхемными отношениями, соответствующая определению декартова произведения для РМД: в результате получается отношение, схема которого включает все атрибуты исходных отношений.

Декартовым произведением отношений $R_{A_1, \dots, A_n}$ и $S_{B_1, \dots, B_k}$ называют отношение $T_{A_1, \dots, A_n, B_1, \dots, B_k}$, содержащее все комбинации кортежей исходных отношений.

R1		R2		
A	B	C	D	E
a1	b1	c1	d1	e1
a2	b2	c2	d2	e2
		c3	d3	e3

R1 × R2

SELECT

admin.id_admin as "администратор",
 weekday.id_weekday as "день недели",
 COUNT(visit.id_visit) as "количество посещений"

FROM Admin

CROSS JOIN Weekday

LEFT JOIN Visit ON Admin.id_admin = Visit.id_admin AND Weekday.id_weekday = visit.id_weekday

GROUP BY admin.id_admin, weekday.id_weekday

ORDER BY admin.id_admin, weekday.id_weekday;

14. Операция объединения. Пример операции.

Объединение (union)

Объединение – бинарная операция над односхемными отношениями.

Объединением отношений $R_{A_1, \dots, A_n}$ и $S_{A_1, \dots, A_n}$ называют отношение

$T_{A_1, \dots, A_n}$, при этом кортеж t_i входит в T , если он принадлежит R или S :

$$t_i \in T: t_i \in R \vee t_i \in S.$$

То есть объединение включает в себя все кортежи обоих исходных отношений без повторов.

```

SELECT visitor_surname, visitor_name, visitor_patronymic
FROM Visitor
WHERE id_visitor IN (
  SELECT DISTINCT pass.id_visitor
  FROM Pass JOIN Visit ON pass.id_pass = visit.id_pass
  WHERE visit.id_weekday = 3
)
UNION
SELECT visitor_surname, visitor_name, visitor_patronymic
  
```



```
FROM Visitor
WHERE id_visitor NOT IN (
    SELECT DISTINCT pass.id_visitor
    FROM Pass JOIN Visit ON pass.id_pass = visit.id_pass
    WHERE visit.id_weekday = 3
)
ORDER BY visitor_surname;
```

15. Операция вычитания. Пример операции.

Вычитание (minus)

Разность – бинарная операция над односхемными отношениями.

Разностью $T = R - S$ отношений $R_{A_1, \dots, A_n}$ и $S_{A_1, \dots, A_n}$ называется множество кортежей R , не входящих в S :

$$t_i \in T: t_i \in R \wedge \text{not}(t_i \in S).$$

```
SELECT visitor_surname, visitor_name, visitor_patronymic
FROM Visitor
EXCEPT
SELECT v.visitor_surname, v.visitor_name, v.visitor_patronymic
FROM Visitor v
JOIN Pass p ON v.id_visitor = p.id_visitor
JOIN Visit vi ON p.id_pass = vi.id_pass
WHERE vi.id_weekday = 3;
```

16. Операция пересечения. Пример операции.

Пересечение (intersect)

Пересечение – бинарная операция над односхемными отношениями.

Пересечением $T = R \cap S$ отношений $R_{A_1, \dots, A_n}$ и $S_{A_1, \dots, A_n}$ называют отношение $T_{A_1, \dots, A_n}$, при этом кортеж t_i входит в T , если он принадлежит обоим исходным отношениям

$$t_i \in T: t_i \in R \vee t_i \in S.$$

Это можно выразить через разность:

$$R \cap S = R - (R - S).$$

```
SELECT v.visitor_surname, v.visitor_name, v.visitor_patronymic
FROM Visitor v
JOIN Pass p ON v.id_visitor = p.id_visitor
JOIN Visit vi ON p.id_pass = vi.id_pass
WHERE vi.id_weekday = 1
```

```
INTERSECT
```

```
SELECT v.visitor_surname, v.visitor_name, v.visitor_patronymic
FROM Visitor v
JOIN Pass p ON v.id_visitor = p.id_visitor
JOIN Visit vi ON p.id_pass = vi.id_pass
WHERE vi.id_weekday = 3;
```

17. Операция соединения. Пример операции.

```
SELECT admin.id_admin, weekday.id_weekday FROM Admin
CROSS JOIN Weekday;
LEFT JOIN Visit ON Admin.id_admin = Visit.id_admin AND Weekday.id_weekday =
visit.id_weekday
```

```
INNER JOIN Pass ON visitor.id_visitor = pass.id_visitor
INNER JOIN Type_of_pass ON pass.id_type_of_pass = type_of_pass.id_type_of_pass
```

```
SELECT * FROM Visit
RIGHT JOIN Admin ON Visit.id_admin = Admin.id_admin;
```


Соединение (join)

Соединение – бинарная операция над разносхемными отношениями. Эта операция определяет подмножество декартова произведения двух разносхемных отношений.

Кортеж декартова произведения входит в результирующее отношение, если выполняется условие соединения F, которое задаёт соотношение значений атрибутов разных таблиц. Соединение может быть выражено так:

$$R \bowtie S = \sigma_F(R \times S)$$

R1

A	B
a1	b1
a2	b2

Если условием является равенство значений двух атрибутов исходных отношений, такая операция называется **эквисоединением**.

Естественным называется эквисоединение по одинаковым атрибутам исходных отношений.

18. Операция деления. Пример операции.

Деление (division)

Деление – это бинарная операция над двумя разносхемными отношениями.

Пусть отношение R содержит атрибуты {r₁,r₂,...,r_k, r_{k+1},...,r_n}, а отношение S – атрибуты {r_{k+1},...,r_n}. Тогда результирующее отношение T содержит атрибуты {r₁,r₂,...,r_k}. Кортеж t_i входит в T, если его декартово произведение с отношением S входит в R:

$$t_i \in T : t_i \times S \subset R.$$

Деление может быть выражено так:

$$R / S = \pi_{r_1, \dots, r_k}(R) - \pi_{r_1, \dots, r_k}((\pi_{r_1, \dots, r_k}(R) \times S) - R).$$

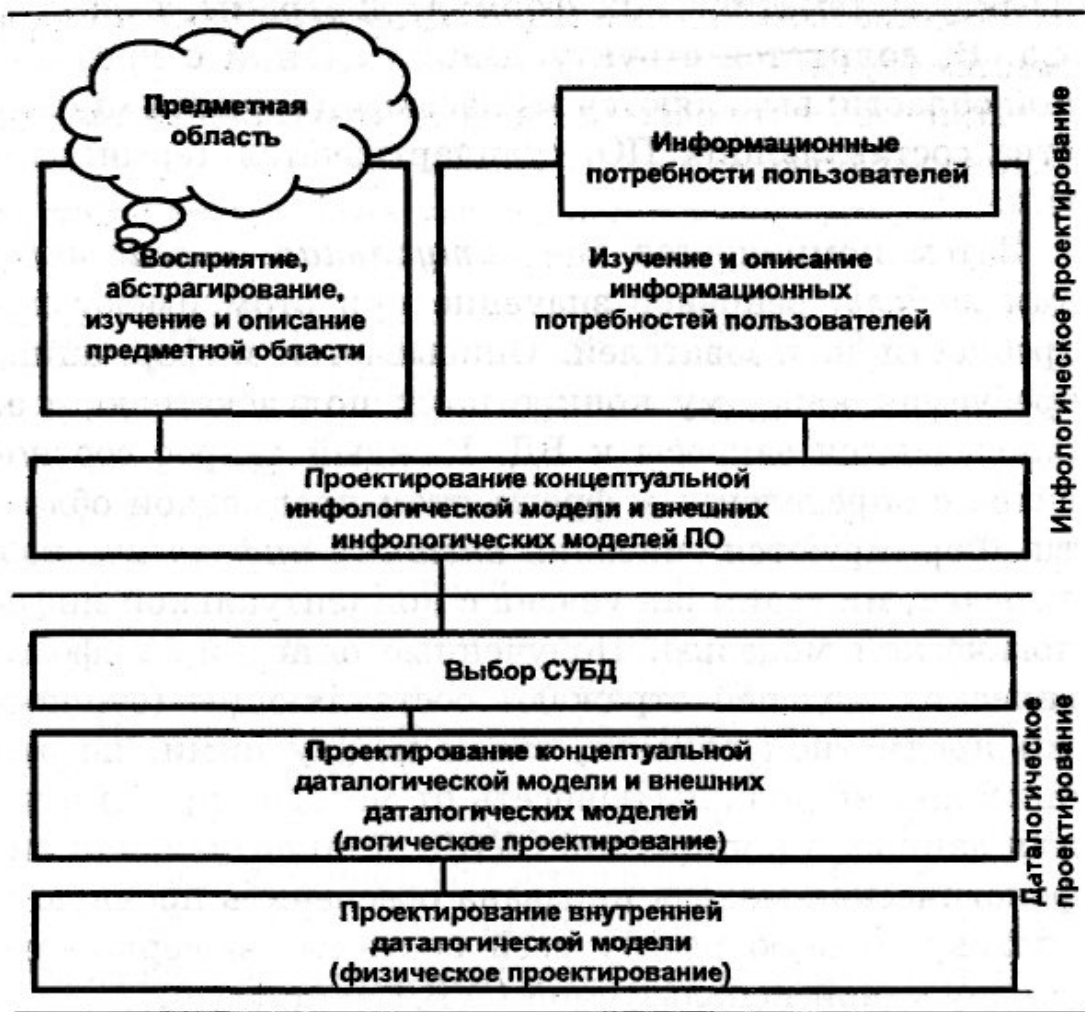
Отношение R				Отношение S		Частное R/S	
A	B	C	D	C	D	A	B
a	b	c	b	c	b	a	b
a	b	g	h	g	h	c	f
c	f	g	h				
c	f	c	b				
a	v	c	b				
c	v	g	h				

```
SELECT v.id_visitor, v.visitor_surname
FROM Visitor v
WHERE NOT EXISTS (
  SELECT w.id_weekday
  FROM Weekday w
  WHERE NOT EXISTS (
    SELECT 1
    FROM Pass p
    JOIN Visit vi ON p.id_pass = vi.id_pass
    WHERE p.id_visitor = v.id_visitor
    AND vi.id_weekday = w.id_weekday
  )
);
```

19. Этапы проектирования и разработки СУБД. Инфологическая, даталогическая и физическая модели данных.

- Инфологическая (концептуальная) модель — описывает предметную область без привязки к СУБД. Это концептуальная модель, которая описывает, какие данные хранятся в системе и какие связи между этими данными существуют, не углубляясь в технические детали реализации. Этот уровень фокусируется на структурировании данных с точки зрения пользователя или предметной области.
- Примеры: ER-диаграммы (схемы сущность-связь), схема объектов (UML-диаграмма), описание предметной области.
- Даталогическая модель представляет собой описание базы данных, выполненное в терминах используемой СУБД, т. е. даталогическая модель описывает собственно данные (информацию, которая будет записана в памяти компьютера) и связи между данными.
- Примеры: схема БД графическая, описание таблиц, лингвистическое описание схемы
- Физическая модель — реализация БД с оптимизацией под конкретное оборудование. Это детализированное представление того, как данные фактически хранятся на физических носителях, например, на жестких дисках. Этот уровень включает конкретные способы хранения и индексирования данных, что позволяет оптимизировать производительность запросов и обеспечить эффективное использование ресурсов.
- Примеры: схема индексов, SQL-код для создания таблиц





20. Средства описания инфологической модели. Результат инфологического моделирования.

Инфологическая модель предметной области (ПрО) включает:

- описание структуры и динамики данных предметной области;
- перечень групп пользователей (ролей) системы;
- описание информационных потребностей групп пользователей.

Методы построения инфологических моделей:

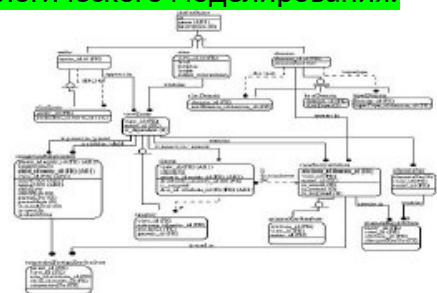
- ориентированные на процессы,
- ориентированные на данные.

Технологии построения инфологических моделей:

- Use Case
- BPM процессы
- IDEF0
- IDEF3
- Диаграммы сущность связь (ER)
- IDEF1x

Результат моделирования:

- ✓ Описание предметной области
- ✓ Концептуальная инфологическая модель.
- ✓ Перечень сущностей
- ✓ Перечень групп пользователей системы.



ER-диаграмма



MyShared

21. Сущности и связи. Графическое представление элементов языка ER-диаграмм.

Сущность (entity) - это объект, который может быть идентифицирован и отличим от других объектов.

Связь (relationship) - это ассоциация, установленная между несколькими сущностями.

Выделяют понятия **тип сущности** и **экземпляр сущности**.

Сущности могут быть:

- базовые,
- зависимые.

У сущности выделяют **атрибуты**

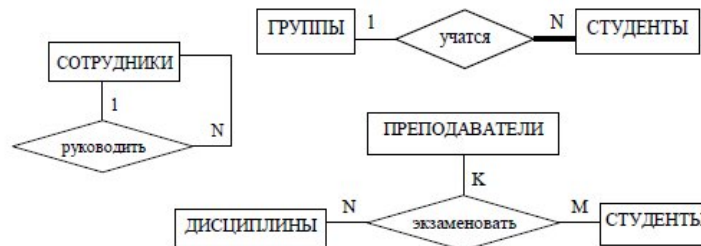
Для каждого атрибута необходимо определить:

- название,
- тип данных,
- ограничения целостности – множество значений, которые может принимать данный атрибут.

Обозначение	Значение
	Набор независимых сущностей
	Набор зависимых сущностей
	Атрибут
	Ключевой атрибут
	Набор связей

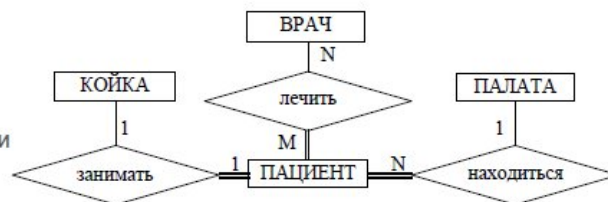
Степень связей между сущностями:

- ✓ **унарная** – связь между разными экземплярами сущностей одного типа
- ✓ **бинарная** – связь между двумя разными типами сущностей
- ✓ **тернарная** – связь между тремя разными типами сущностей



Связи между сущностями

- название,
- тип (факультативная или обязательная),
- **кардинальность** (1:1, 1:n или m:n),
- **степень** (унарная, бинарная, тернарная или n-арная).



Нотация Чена: прямоугольники, ромбы, овалы, прямые линии

22. Проектирование даталогической модели. Аномалии. Типы аномалий. Пример.

Даталогическое проектирование сводится к представлению инфологической модели в терминах выбранной системы управления базами данных (СУБД), т. е. даталогическая модель описывает собственно данные (информацию, которая будет записана в памяти компьютера) и связи между данными.

В системах баз данных даталогическая модель предметной области представляет собой схему базы данных.

Аномалией называется такая ситуация в таблице БД, которая приводит к противоречию в БД или существенно усложняет обработку БД.

Источник аномалий – повторение данных в таблицах.

Решение – уменьшение повторений атрибутов сущностей
Идеально – одна сущность повторяется один раз

CatID	CatName	Product1	Product2	Product3
1	бытовая химия	порошок	мыло	сода
2	продукты	сода	-	-
3	спорт. инвентарь	лыжи	палки	-

Аномалии:

1. Обновления - изменение одних данных требует просмотра всей таблицы и изменения некоторых записей.
Пример: замена «сода» на «сода пищевая».
2. Удаления — при удалении какого либо кортежа из таблицы удаляются прочие данные.
Пример: удаление «сода» удалит два продукта.
3. Добавления – нельзя добавить данные, пока нет всего набора данных, иначе вставка записи требует просмотра всей таблицы.
Пример: нельзя добавить продукт «квадрокоптер» пока не создана категория «летательные аппараты» и «игрушки».

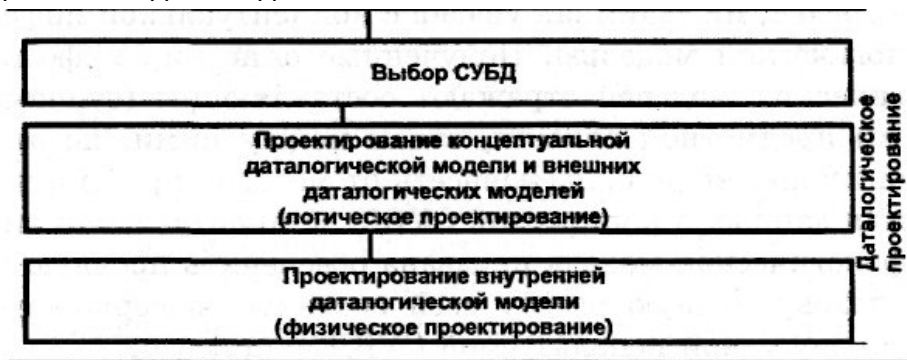
Требуется реорганизовать хранение данных так, чтобы аномалии не проявлялись.

Формализм нормальных форм основанный на функциональных зависимостях

Процедура нормализации

Этапы проектирования даталогической модели:

Описание таблиц базы данных, лингвист описание схемы, графическое представление схемы базы данных, создание бд



23. Понятие функциональной зависимости. Особенности функциональной зависимости. Пример функциональной зависимости.

Определение: Функциональная зависимость $X \rightarrow Y$ называется полной, если атрибут Y не зависит функционально от любого подмножества X . (-потенциальный ключ)

Неключевой атрибут функционально полно зависит от составного ключа, если он функционально зависит от ключа, но не находится в функциональной зависимости ни от какой части составного ключа.

Пусть R – переменная отношения, X, Y – атрибуты R . Тогда:

- Y функционально зависит от X , или
- X функционально определяет Y , или
- X является детерминантом, а Y является зависимой частью
- $X \rightarrow Y$

если не существует двух различных кортежей, которые в атрибуте X имеют совпадающие значения, а в атрибуте Y имеют разные значения

КодП	ИмяП	Город	Рейтинг
S1	Бендер	Черноморск	15
S2	ИТ-монстр	Москва	29
S3	Разноэкспорт	Черноморск	15
S4	Макрохард	Москва	29
...	...	...	...

$\{\text{Город}\} \rightarrow \{\text{Рейтинг}\}$

Особенности ФЗ:

1. Определяются на домене. ФЗ формулируются на всех значениях всех возможных кортежей
2. Определяются семантикой. ФЗ определяются из предметной области, и не выводятся из состояния БД.

ФЗ – ограничение целостности

Частный случай: Функциональная зависимость от первичного ключа, по определению первичного ключа.

$\{\text{КодП}\} \rightarrow \{\text{ИмяП}, \text{Город}, \text{Рейтинг}\}$

Чем меньше ФЗ – тем проще работа СУБД

КодП	ИмяП	Город	Рейтинг
S1	Бендер	Черноморск	15
S2	ИТ-монстр	Москва	29
S3	Разноэкспорт	Черноморск	15
S4	Макрохард	Москва	29
...	...	...	...

Пусть:

R – отношение, A, B, C – атрибуты R .

Выполняется ФЗ $A \rightarrow B$

$$R = \pi_{A,B}(R) \bowtie \pi_{A,C}(R)$$



$A \rightarrow B$

A, B, C – могут быть составными.

Пример:

C		A		B
КодП	ИмяП	Город	Рейтинг	
S1	Бендер	Черноморск	15	
S2	ИТ-монстр	Москва	29	
S3	Разноэкспорт	Черноморск	15	
S4	Макрохард	Москва	29	
...	...	...	...	

$\{\text{Город}\} \rightarrow \{\text{Рейтинг}\}$

$\pi_{\{\text{Город}, \text{Рейтинг}\}}(R)$	
Город	Рейтинг
Черноморск	15
Москва	29
...	...

$\pi_{\{\text{КодП}, \text{ИмяП}, \text{Город}\}}(R)$		
КодП	ИмяП	Город
S1	Бендер	Черноморск
S2	ИТ-монстр	Москва
S3	Разноэкспорт	Черноморск
S4	Макрохард	Москва
...	...	...

Теорема Хита обеспечивает декомпозицию отношений без потерь данных

Входное отношение с ФЗ преобразуется в другие отношения, в каждом из которых атрибуты минимально зависят от первичного ключа.

Атрибут B минимально зависит от атрибута A , если выполняется минимальная слева ФЗ $A \rightarrow B$.

Пример: зная `admin_id` можно однозначно найти его ФИО и опыт работы. Обратная зависимость неверна

Теорема Хита утверждает, что если некоторая декомпозиция выполняется в соответствии с определенной функциональной зависимостью, то она будет выполнена без потерь.

24. Структурированный язык управления запросами. Состав. Достоинства и недостатки.

Реляционная алгебра – определяет возможность управления объектами.

Реляционное исчисление – формулировка запроса к базе данных безотносительно реализации.

SQL – язык запросов к базам данных, с реализацией в конкретной СУБД.

1970 Прототип языка SQL – (SEQUEL)
IBM Research System R.
1986 Стандарт ANSI - SQL-86 SQL-87
1992 SQL-92
2008 OLAP-функций
2016 JSON

Реляционная алгебра, реляционное исчисление и SQL – декларативные конструкции

Состав типичного SQL:

- DDL (язык определения данных) состоит из команд которые управляют объектами (таблицы, индексы, просмотры, и так далее) в базе данных.
- DML (язык манипулирования данными) - это набор команд которые управляют значениями таблиц.
- DCL (язык доступа к данным) – проверяет права доступа к данным
- TCL (язык управления транзакциями) - определяет права совместного доступа к данным.
- Программный SQL.

Преимущества:

- независимость от конкретной СУБД
- стандартизация
- декларативность

Недостатки

- несоответствие классической РМД;
- Таблица ≠ отношение, строка ≠ кортеж
- сложность понимания
- Избыточность языковых средств
- Большой объем стандарта
- Отступления от стандартов
- "Надмножества подмножества SQL": Oracle PL/SQL, MS TransactSQL, PL/pgSQL и др.

25. Язык описания данных и язык определения данных. Взаимосвязь языков. Схема.

Администратор БД – управляет схемой базы данных

Программист (прикладная программа) – управляет данными в созданной базе данных

Сущности:

DDL (язык определения данных):

- базы данных (схемы)
- таблицы
- индексы

DML (язык манипулирования данными):

- таблица

Операции:

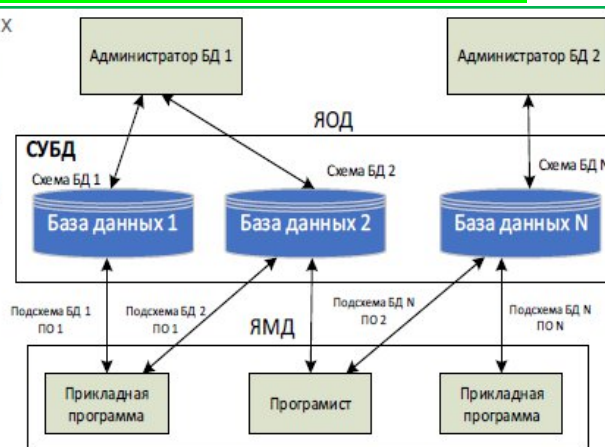
- создание
- изменение
- удаление
- просмотр

DDL

- database ➢ create
- table ➢ alter
- index ➢ drop

DML

- table ➢ delete
- select
- Insert
- update



Администратор БД – управляет метаданными

Программист – управляет данными

Нормальная форма – совокупность ограничений, накладываемых на схему отношения. Ограничения исключают аномалии при работе с базой данных.

Нормализация схемы отношения выполняется путём декомпозиции схемы.

Декомпозицией схемы отношения R называется замена её совокупностью схем отношений A_i таких, что:

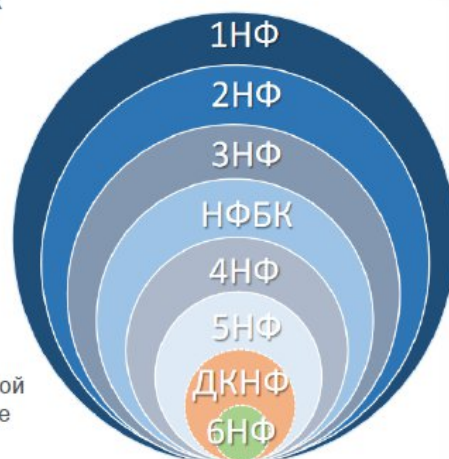
$$R = \bigcup_i A_i$$

Не требуется, чтобы отношения A_i были не пересекающимися.

Условия:

1. Декомпозиция отношения не должна приводить к потере зависимостей между атрибутами сущностей.
2. Для декомпозиции должна существовать операция реляционной алгебры, применение которой позволит восстановить исходное отношение.

Нормализация состоит в получении из исходного набора отношений нормализованных отношений, каждое из которых находится в НФ как можно более высокого порядка



Определение: Переменная отношения R находится в первой нормальной форме (1НФ) тогда и только тогда, когда в любом допустимом значении отношения каждый его кортеж содержит только одно значение для каждого из атрибутов.

Простой атрибут

- Название
- Дата
- Количество
- Фамилия

Составной атрибут

- ФИО
- Адрес
- Контактные данные

Поставки

Код_П	Имя_П	Город_П	Рейтинг	Поставка
S1	Бендер	Черноморск	15	P1, P2, P13
S3	ИТ-монстр	Москва	29	P5, P1

Не 1НФ!



Неформальное определение:

1. Все строки должны быть различными.
2. Все элементы внутри ячеек должны быть атомарными (не списками). Другими словами, элемент является атомарным, если его нельзя разделить на части, которые могут использовать в таблице независимо друг от друга.

Простой атрибут – это атрибут, значения которого атомарны (неделимы).

Сложный атрибут – это составной и/или многозначный атрибут. Составной атрибут имеет значение, которое является конкатенацией нескольких значений одного или разных доменов.

Любое реляционное отношение находится в 1НФ (по определению реляционного отношения).

Для того чтобы привести к 1НФ отношение, содержащее сложные атрибуты, нужно:

- разделить составные атрибуты на простые;
- построить декартово произведение всех многозначных атрибутов с кортежами, к которым они относятся.

В отношениях, полученных после приведения к 1НФ, первичный ключ (ПК) состоит из первоначального ПК и всех многозначных атрибутов.

Определение: Отношение находится во 2НФ, если оно приведено к 1НФ и каждый неключевой атрибут функционально полностью зависит от составного ключа отношения.

Неформальное определение:

1. Таблица должна находиться в первой нормальной форме.
2. Любой атрибут, не входящий в состав первичного ключа, функционально зависит только от всех атрибутов ключа и не зависит от его отдельных атрибутов.

Для того чтобы привести к 2НФ отношение, содержащее сложные атрибуты, нужно:

- построить его проекцию, исключив атрибуты, которые не находятся в функционально полной зависимости от составного ключа.
- построить дополнительные проекции на часть составного ключа и атрибуты, функционально зависящие от этой части ключа.

Определение: Пусть X, Y, Z – атрибуты отношения R. Функциональная зависимость $X \twoheadrightarrow Z$ называется **транзитивной**, если $X \twoheadrightarrow Y$ и $Y \twoheadrightarrow Z$, но не $Z \twoheadrightarrow X$ и $Y \twoheadrightarrow X$.

Определение: Отношение находится во 3НФ, если оно приведено к 2НФ и в нем отсутствуют транзитивные зависимости.

Хранение

Поставка
S1
S5
S3
S2

Неформальное определение:

1. Таблица должна находиться во второй нормальной форме.
2. Любой ее неключевой атрибут функционально зависит только от первичного ключа.

Для того чтобы привести к 3НФ отношение, содержащее сложные атрибуты, нужно:

- построить проекцию, исключив транзитивно зависящие от ПК атрибуты;
- построить дополнительно одну или несколько проекций на детерминанты исходного отношения и атрибуты, функционально зависящие от них.

26. Типы данных в СУБД. Классификация. Типы данных, ориентированные на человека или машину.

Примеры.

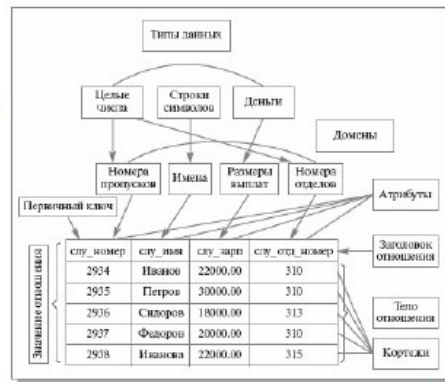
- | | | | |
|------------------------------|----------------------|------------------|--------------------------|
| Типы данных | Охват доменов | Размер данных: | Ориентированные на: |
| ➢ Числовые типы данных | ➢ универсальные | ➢ Большие объемы | ➢ Вычислительную систему |
| ➢ Типы данных даты и времени | ➢ Специализированные | ➢ Малые объемы | ➢ Человека |
| ➢ Символьные типы данных | | | |
| ➢ Двоичные данные | | | |
| ➢ Логические | | | |

Значения данных, хранимые в реляционной базе данных, являются типизированными, т. е. известен тип каждого хранимого значения.

Понятие типа данных в реляционной модели данных полностью соответствует понятию типа данных в языках программирования.

Определение *типа данных* состоит из трех компонентов:

- определение множества значений данного типа;
- определение набора операций,
- применимых к значениям типа;
- определение способа внешнего представления значений типа (литералов).



14

Числа: tinyint, smallint, mediumint, int, bigint, float, double, decimal(size, d)

Текст: char, varchar, tinytext, text, blob (двоичные большие объекты), mediumtext, mediumblob, longtext, longblob, enum, set

Даты: date, datetime, timestamp, time, year

27. Операторы управления базой данных (схемой).

Создание:

```
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] db_name
[create_option] ...
```

```
create_option: [DEFAULT] {
    CHARACTER SET [=] charset_name
    | COLLATE [=] collation_name
    | ENCRYPTION [=] {'Y' | 'N'}
}
```

Изменение:

```
ALTER {DATABASE | SCHEMA} [db_name]
alter_option ...
```

```
alter_option: {
    [DEFAULT] CHARACTER SET [=] charset_name
    | [DEFAULT] COLLATE [=] collation_name
    | [DEFAULT] ENCRYPTION [=] {'Y' | 'N'}
    | READ ONLY [=] {DEFAULT | 0 | 1}
}
```

Удаление:

```
DROP {DATABASE | SCHEMA} [IF EXISTS] db_name
```

DATABASES | SCHEMAS одно и то же!

CHARACTER SET – кодовая страница

```
mysql> SHOW CHARACTER SET LIKE 'utf%';
```

Charset	Description	Default collation	Maxlen
utf16	UTF-16 Unicode	utf16_general_ci	4
utf16le	UTF-16LE Unicode	utf16le_general_ci	4
utf32	UTF-32 Unicode	utf32_general_ci	4
utf8mb3	UTF-8 Unicode	utf8mb3_general_ci	3
utf8mb4	UTF-8 Unicode	utf8mb4_0900_ai_ci	4

COLLATION– способ сортировки

```
mysql> SHOW COLLATION WHERE Charset = 'utf8mb4';
```

collation	Charset	Id	default	compiled	sortlen	pad_attribute
utf8mb4_0900_ai_ci	utf8mb4	255	Yes	Yes	0	NO PAD
utf8mb4_0900_as_ci	utf8mb4	305		Yes	0	NO PAD

Просмотр списка созданных

```
SHOW {DATABASES | SCHEMAS}
```

```
[LIKE 'pattern' | WHERE expr]
```

Просмотр создания:

```
SHOW CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] db_name
```

28. Операторы управления таблицей. Пример создания таблицы.

Создание:

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
(create_definition,...)
[table_options]
[partition_options]
```

```
create_definition: {
    col_name column_definition
    | {INDEX | KEY} [index_name] [index_type] (key_part,...)
    [index_option] ...
    | {FULLTEXT | SPATIAL} [INDEX | KEY] [index_name] (key_part,...)
    [index_option] ...
    | [CONSTRAINT [symbol]] PRIMARY KEY
    [index_type] (key_part,...)
    [index_option] ...
    | [CONSTRAINT [symbol]] UNIQUE [INDEX | KEY]
    [index_name] [index_type] (key_part,...)
    [index_option] ...
    | [CONSTRAINT [symbol]] FOREIGN KEY
    [index_name] (col_name,...)
    reference_definition
    | check_constraint_definition
}
```

CREATE TABLE IF NOT EXISTS `swimming\_pool` `

```
Type_of_the_day ` (
`id_day` INT PRIMARY KEY NOT NULL AUTO_INCREMENT ,
`type_of_day` VARCHAR (8) NOT NULL DEFAULT 'будний');
```


Изменение:

```
ALTER TABLE tbl_name
[alter_option [, alter_option] ...]
[partition_options]

alter_option: {
    table_options
  | ADD [COLUMN] col_name column_definition
    [FIRST | AFTER col_name]
  | ADD [COLUMN] (col_name column_definition,...)
  | ADD {INDEX | KEY} [index_name]
    [index_type] (key_part,...) [index_option] ...
  | ADD {FULLTEXT | SPATIAL} [INDEX | KEY] [index_name]
    (key_part,...) [index_option] ...
  | ADD [CONSTRAINT [symbol]] PRIMARY KEY
    [index_type] (key_part,...)
    [index_option] ...
  | ADD [CONSTRAINT [symbol]] UNIQUE [INDEX | KEY]
    [index_name] [index_type] (key_part,...)
    [index_option] ...
  | ADD [CONSTRAINT [symbol]] FOREIGN KEY
    [index_name] (col_name,...)
    reference_definition
  | ADD [CONSTRAINT [symbol]] CHECK (expr) [[NOT] ENFORCED]
  | DROP {CHECK | CONSTRAINT} symbol
  | ALTER {CHECK | CONSTRAINT} symbol [NOT] ENFORCED
  | ALGORITHM [=] {DEFAULT | INSTANT | INPLACE | COPY}
  | ALTER [COLUMN] col_name {
    SET DEFAULT {literal | (expr)}
  | SET {VISIBLE | INVISIBLE}
  | DROP DEFAULT
}
```

```
| ALTER INDEX index_name {VISIBLE | INVISIBLE}
| CHANGE [COLUMN] old_col_name new_col_name column_definition
    [FIRST | AFTER col_name]
| [DEFAULT] CHARACTER SET [=] charset_name [COLLATE [=] collation_name]
| CONVERT TO CHARACTER SET charset_name [COLLATE collation_name]
| {DISABLE | ENABLE} KEYS
| {DISCARD | IMPORT} TABLESPACE
| DROP [COLUMN] col_name
| DROP {INDEX | KEY} index_name
| DROP PRIMARY KEY
| DROP FOREIGN KEY fk_symbol
| FORCE
| LOCK [=] {DEFAULT | NONE | SHARED | EXCLUSIVE}
| MODIFY [COLUMN] col_name column_definition
    [FIRST | AFTER col_name]
| ORDER BY col_name [, col_name] ...
| RENAME COLUMN old_col_name TO new_col_name
| RENAME {INDEX | KEY} old_index_name TO new_index_name
| RENAME [TO | AS] new_tbl_name
| {WITHOUT | WITH} VALIDATION
```

Удаление:

```
DROP [TEMPORARY] TABLE [IF EXISTS]
tbl_name [, tbl_name] ...
[RESTRICT | CASCADE]
```

Удаление – происходит сразу и навсегда!

Просмотр колонок таблицы

```
SHOW [EXTENDED] [FULL] {COLUMNS | FIELDS}
{FROM | IN} tbl_name
[{FROM | IN} db_name]
[LIKE 'pattern' | WHERE expr]
```

```
mysql> SHOW COLUMNS FROM City;
```

Field	Type	Null	Key	Default	Extra
ID	int(11)	NO	PRI	NULL	auto_increment
Name	char(35)	NO			
CountryCode	char(3)	NO	MUL		
District	char(20)	NO			
Population	int(11)	NO		0	

Просмотр создания таблицы

```
SHOW CREATE TABLE tbl_name
```

```
CREATE TABLE IF NOT EXISTS `state` (
  `id_State` int(10) unsigned NOT NULL AUTO_INCREMENT,
  `State` varchar(20) COLLATE utf8_bin DEFAULT NULL,
  PRIMARY KEY (`id_State`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_bin AUTO_INCREMENT=2;
```

Просмотр списка таблиц

```
SHOW [EXTENDED] [FULL] TABLES
[{FROM | IN} db_name]
[LIKE 'pattern' | WHERE expr]
```

```
-----+-----
Tables_in_database_name |
-----+-----
actions                  |
permissions              |
permissions_roles        |
permissions_users        |
roles                    |
roles_users              |
settings                 |
users                    |
-----+-----
8 rows in set (0.00 sec)
```

29. Понятие ссылочной целостности References. Действия при проверке ссылочной целостности.

Объявление ссылочной целостности:

```
[CONSTRAINT [symbol]] FOREIGN KEY
[index_name] (col_name, ...)
REFERENCES tbl_name (col_name, ...)
[ON DELETE reference_option]
[ON UPDATE reference_option]
```

Действия при ссылочной целостности:

```
reference_option:
    RESTRICT | CASCADE | SET NULL | NO ACTION | SET DEFAULT
```

CASCADE. Удаление записи из родительской таблицы приводит к удалению записей с соответствующим значением внешнего ключа

NO ACTION, RESTRICT. Удаление строки из родительской таблицы запрещается, если в дочерней таблице существует хотя бы одна ссылающаяся на нее строка.

SET NULL. При удалении строки из родительской таблицы во всех ссылающихся на нее строках дочернего отношения в поле внешнего ключа, соответствующего первичному ключу удаленной строки, записывается пустое значение.

SET DEFAULT. При удалении строки из родительской таблицы в поле внешнего ключа всех ссылающихся на нее строк дочерней таблицы автоматически помещается значение, указанное для этого поля как значение по умолчанию.

Address (адрес)

Key	House	Apart	Street
12	52	1	15
15	12-а	2	15
85	9	3	NULL
152	3	4	120
254	85-6	5	122
374	132	6	150
495	2	7	35
514	52	8	15
632	75	9	130
887	56	10	155
994	47	11	12
1024	32	12	84
4511	14	13	150

Street (улица)

Key	Prefix	Name
15	ул	Ленина
35	ул	Энгельса
84	ул	Дзержинского
120	ул	Клары Цеткин
122	ул	Розы Люксембург
130	пр	Московский
150	пр	Победы

30. Средства обеспечения целостности данных В СУБД.

Целостность данных — это корректность и непротиворечивость информации в базе данных. СУБД предоставляет несколько механизмов для ее поддержания:

- Primary key гарантирует уникальность и отсутствие NULL в столбце
- Foreign key обеспечивает ссылочную целостность: значение в одном столбце должно существовать в другом
- Unique запрещает дублирование значений (разрешает NULL)
- Check ограничивает допустимые значения в столбце
- NOT NULL
- Триггеры: автоматически выполняют код при событиях insert, update, delete
- Cascade автоматически обновляет или удаляет связанные данные
- Транзакции группируют несколько операций в атомарное действие

Обеспечение целостности данных

Типы ограничений целостности в языке SQL:

1. Уникальность значения первичного ключа (PRIMARY KEY). 1,2
явные структурные
ограничения
целостности
2. Уникальность ключевого поля или комбинации значений ключевых полей: UNIQUE(A), где A – один или несколько атрибутов, указанных через запятую.
3. Обязательность/необязательность значения (NOT NULL / NULL).
4. Задание диапазона значений атрибута: CHECK(field BETWEEN min_value AND max_value)
5. Задание взаимоотношений между значениями атрибутов Field1 и Field2: CHECK (field1 @ field2), где @ – оператор отношения (например, знак ">").
6. Задание списка возможных значений (констант) для атрибута Field: CHECK (field IN (value1, value2,..., valueN)). 4-8
явные ограничения
целостности
значений
7. Определение формата атрибута Field с помощью фрагмента LIKE. Пример: CHECK (field LIKE '____-____')
8. Определение домена атрибута на основе значений другого атрибута: FOREIGN KEY.
9. Ограничения между операциями (В СУБД не реализованы требуют процедур, функций или триггеров) ограничения динамической
целостности
10. Ограничения на параллельное выполнение операций ограничения совместного использования

31. Язык манипуляции данными. Перечень операторов.

INSERT – добавление строк в таблицу.

- Добавляет одну или несколько строк в указанную таблицу.

DELETE – удаление строк из таблицы.

- Удаляет одну или несколько строк из таблицы.
- Можно указать условие, по которому выбираются удаляемые строки.
- Если условие не указано, удаляются все строки таблицы.
- Если ни одна строка не удовлетворяет условию, ни одна строка не будет удалена.

SELECT – выбор данных из таблиц в память.

- Однотабличные запросы.
- Многотабличные запросы соединением данных.
- Агрегатные функции.
- Группировка данных.
- Сортировка данных.
- Вложенные запросы.

UPDATE – изменение данных.

- Изменяет значения одного или нескольких полей в записях указанной таблицы.
- Можно указать условие, по которому выбираются обновляемые строки.
- Если условие не указано, обновляются все строки таблицы.
- Если ни одна строка не удовлетворяет условию, ни одна строка не будет обновлена.

Части запроса:

- SELECT – выполнение проекции.
- FROM – перечисление отношений и связей.
- WHERE – выполнение выборки.
- GROUP BY – группировка данных.
- ORDER BY – сортировка данных.
- LIMIT – ограничения числа вывода.
- HAVING – постобработка результата.

Общий алгоритм выполнения операции SELECT

Having Total > 50
Order By Total;

1. Статический синтаксический анализ конструкций (Select, from, where).
2. Динамический анализ принадлежности атрибутов таблицам (уникальность, правильность имени)
3. Проверка прав доступа к таблицам и атрибутам.
4. Построение плана запроса (построение последовательностей соединения таблиц).
5. Проверка для каждой строки каждой таблицы условия отбора (where).
6. Извлечение из полученных записей тех полей, которые заданы в списке вывода (select), и формирование ответа.
7. Выполнение соединения таблиц (вложенных запросов).
8. Сортировка для получения группировки.
9. Группировка полученных в результате отбора записей (group by)
10. Вычисление для этих групп значений агрегирующих функций.
11. Если присутствуют конструкции UNION, INTERSECT, MINUS. Повторить п.п.1-10 для каждого запроса.
12. Соединить по правилу. п.11. результаты запросов.
13. Выбор тех групп, которые удовлетворяют условию отбора групп (having).
14. Сортировка полученных записей в указанном порядке (order by).

32. Оператор добавления данных. Два варианта реализации. Пример.

INSERT – добавление строк в таблицу

```
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name
[PARTITION (partition_name [, partition_name] ...)]
[(col_name [, col_name] ...)]
{ {VALUES | VALUE} (value_list) [, (value_list)] ... }
[AS row_alias[(col_alias [, col_alias] ...)]]
[ON DUPLICATE KEY UPDATE assignment_list]
```

value_list:
value [, value] ...

```
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name
[PARTITION (partition_name [, partition_name] ...)]
SET assignment_list
[AS row_alias[(col_alias [, col_alias] ...)]]
[ON DUPLICATE KEY UPDATE assignment_list]
```

assignment:
col_name =
value

INSERT INTO имя_таблицы (список_полей_таблицы) VALUES (список_значений);

INSERT INTO имя_таблицы SET атрибут1 = значение1, атрибут2 = значение2;

INSERT INTO 'subscriber' VALUES('Яцко', 'Альберт',
'Феликсович', 19940626, '+7(985) 506-96-57', '106196', 'кп.сч.
10001212312312301231230', 'Районные комбинаты социального
питания', 'СПБ', 1199488240);

INSERT INTO 'subscriber' Set 'surname' = 'Яцко', 'name' = 'Альберт',
'patronymic' = 'Феликсович', 'date_of_birth' = 19940626,
'phone_for_communication' = '+7(985) 506-96-57', 'Bank_details' = 'кп.сч.
10001212312312301231230', 'workplace' = 'Районные комбинаты
социального питания', 'place_of_residence' = 'СПБ',
'passport' = '2992730765';

Значение по умолчанию используется только тогда, когда значение поля не вводится в явном виде.

Update – добавление строк в таблицу

```
UPDATE [LOW_PRIORITY] [IGNORE] table_reference
SET assignment_list
[WHERE where_condition]
[ORDER BY ...]
[LIMIT row_count]
```

assignment:
col_name = value

UPDATE – изменение данных:

UPDATE имя_таблицы

SET имя_поля1 = выражение1 [, имя_поля2 = выражение
[WHERE условие];

Если забыть where – заменятся все строки в таблице

WHERE – условие выборки

Содержит пары «имя атрибута» «значение атрибута»

Пары соединяются знаками (>, <, <=, >=, !=)

Конструируется по правилам логического выражения: and,
or, !, (,).

Используются специальные синтаксические конструкции:

- > IN,
- > BETWEEN,
- > LIKE
- > IS [NOT] NULL.

WHERE – примеры

ID = 5 Date > '2002-12-12' Cost <= 100

(Date > '2002-12-12') and (Cost <= 100)

(Date > '2002-12-12') or (Cost <= 100)

ID IN (3,5,100) ID IN (Select)

Date BETWEEN '2002-12-12' AND '2002-12-18.

Like 'a%' Like '_a'

' ' – один любой символ,

'%' – произвольное количество любых символов (или ни одного).

Update 'subscriber' Set 'surname' = 'Петров' where 'passport' = '2992730765';

Update 'subscriber' Set 'surname' = 'Петров' where 'id' = 112122;

33. Оператор удаления данных. Элемент Where. Пример.

Delete – удаление строк из таблицы

```
DELETE [LOW_PRIORITY] [QUICK] [IGNORE] FROM tbl_name [[AS] tbl_alias]
[PARTITION (partition_name [, partition_name] ...)]
[WHERE where_condition]
[ORDER BY ...]
[LIMIT row_count]
```

Если забыть where – удаляться все строки в таблице

DELETE FROM имя_таблицы [WHERE условие];

ORDER BY – сортировка

ASC – по возрастанию

DESC – по убыванию

Limit – выбор заданного числа записей

Limit n – первых n записей

Limit n,m – первых n записей

DELETE 'subscriber' where 'passport' = '2992730765';

DELETE 'subscriber' where 'id' = 112122;

DELETE 'subscriber' order by 'date_of_birth' DESC limit 1;

В информационной системе
удаление крайне нежелательная
операция!

34. Оператор изменения данных. Пример.

UPDATE – изменение данных.

- Изменяет значения одного или нескольких полей в записях указанной таблицы.
- Можно указать условие, по которому выбираются обновляемые строки.
- Если условие не указано, обновляются все строки таблицы.
- Если ни одна строка не удовлетворяет условию, ни одна строка не будет обновлена.

--9. Добавьте первому сотруднику

дополнительный номер телефона.

UPDATE "Persons"."EMPLOYEE"

SET phone = array_append(phone, '+79167777777')

WHERE id = 1;

35. Части оператора выборки данных.

[LIMIT row_count]

assignment:

col_name = value

WHERE – условие выборки

Содержит пары «имя атрибута» «значение атрибута»

Пары соединяются знаками (>, <, <=, >=, !=)

Конструируется по правилам логического выражения: and, or, ! ().

Используются специальные синтаксические конструкции:

- IN,
- BETWEEN,
- LIKE
- IS [NOT] NULL.

Если забыть where – заменятся все строки в таблице

WHERE – примеры

ID = 5 Date > '2002-12-12' Cost <= 100

(Date > '2002-12-12') and (Cost <= 100)

(Date > '2002-12-12') or (Cost <= 100)

ID IN (3,5,100) ID IN (Select)

Date BETWEEN '2002-12-12' AND '2002-12-18.

Like 'a%' Like '_a'

' ' – один любой символ,

'%' – произвольное количество любых символов (или ни одного).



ЯМД. КОМАНДА SELECT

ПОЛИТЕХ

Select – запрос на выборку данных

SELECT

```
[ALL | DISTINCT | DISTINCTROW ]
[HIGH_PRIORITY]
[STRAIGHT_JOIN]
[SQL_SMALL_RESULT] [SQL_BIG_RESULT] [SQL_BUFFER_RESULT]
[SQL_NO_CACHE] [SQL_CALC_FOUND_ROWS]
select_expr [, select_expr] ...
[into_option]
[FROM table_references
[PARTITION partition_list]]
[WHERE where_condition]
[GROUP BY {col_name | expr | position}, ... [WITH ROLLUP]]
[HAVING where_condition]
[WINDOW window_name AS (window_spec)
[, window_name AS (window_spec)] ...]
[ORDER BY {col_name | expr | position}
[ASC | DESC], ... [WITH ROLLUP]]
[LIMIT [{offset}, row_count | row_count OFFSET offset]]
[into_option]
[FOR {UPDATE | SHARE}
[OF tbl_name [, tbl_name] ...]
[NOWAIT | SKIP LOCKED]
| LOCK IN SHARE MODE]
[into_option]
```

Общий синтаксис:

SELECT { [ALL | DISTINCT] } { список_вывода | * }
FROM имя_таблицы1 [алиас1] [, имя_таблицы2 [алиас2] ,...]

[WHERE условие_отбора_записей]

[GROUP BY { имя_поля | выражение },...]

[HAVING условие_отбора_групп]

[ORDER BY имя_поля1 [целое [ASC | DESC]

[, имя_поля2 [целое [ASC | DESC] ,...]];

Общий синтаксис списка вывода:

[{ all | distinct }] { * | выражение1 [алиас1] [, выражение2 [алиас2] ,...] }

Общий синтаксис списка вывода:

ALL – выбор всех записей

DISTINCT – исключение повторяющихся значений

* - выбор всех атрибутов – **крайне нежелательно!**

Alias – псевдоним нужен для формирования нового имени атрибута или отношения.

36. Однотабличная выборка данных по условию. Пример.

Выдать список названий и цен деталей не из Парижа с весом более 10.

Select P.Name, P.Price
From P
Where P.City <> "Париж"
and P.Weight > 10;

PID	Name	City	Weight	Price
P1	Болт	Париж	15	72
P34	Шуруп	Москва	29	108
P22	Гайка	Челябинск	20	18
P45	Шуруп	Одесса	14	108



Результат

Name	Price
Шуруп	108
Гайка	18
Шуруп	108

37. Агрегатные функции в запросах на выборку. Группировка в агрегатных функциях. Пример.

Агрегатная функция – выполняет вычисление на наборе значений и возвращает одиночное значение.

Часто используемые агрегатные функции:

- **AVG**: вычисляет среднее значение. Применяется к полям числового типа.
- **SUM**: вычисляет сумму значений. Применяется к полям числового типа.
- **MIN**: вычисляет наименьшее значение. Применяется к полям любого типа.
- **MAX**: вычисляет наибольшее значение. Применяется к полям любого типа.
- **COUNT**: вычисляет количество строк в запросе. Применяется к полям любого типа.
 - count (*) – количество строк результата;
 - count (имя_поля) – количество значений указанного поля, не являющихся NULL-значениями.
 - count (distinct имя_поля) – количество разных не-NULL значений указанного поля.

Обычно применяются с группировкой **GROUP BY**

Получить код и общее количество поставки каждой детали с числом более 100

Select SP.PID, sum(SP.QTY) as Total,
From SP
Group by SP.PID
Having Total > 100;

Select SP.PID, sum(SP.QTY) as Total,
From SP
Where Total > 100
Group by SP.PID;

Запрос не будет выполнен

SP

SID	PID	Qty
S1	P5	10
S3	P2	20
S2	P4	30
S7	P1	40
S5	P5	20
S5	P3	50
S8	P2	30
S4	P1	80

Создание групп по полю SP.PID

SID	PID	Qty
S1	P5	10
S5	P5	20
S3	P2	20
S8	P2	30
S2	P4	30
S7	P1	40
S4	P1	80
S5	P3	50

упорядочивание

SID	PID	Qty
S1	P5	10
S5	P5	20
S3	P2	20
S8	P2	30
S2	P4	30
S7	P1	40
S4	P1	80
S5	P3	50

выделение групп по полю SP.PID

Применение HAVING

PID	Total
P5	30
P2	50
P4	30
P1	120
P3	50

PID	Total
P1	120

38. Многотабличные запросы. Виды соединений. «Вертикальное» и «Горизонтальное» виды соединения.

Соединение таблиц – создание нового отношения путем объединения двух других отношений по правилам реляционного исчисления.

Соединения осуществляются по ключам с помощью соединений join

1. Декартово произведение А и В **CROSS JOIN, JOIN** (без ON)
Select <...> From A CROSS JOIN B;

2. Соединение при совпадении ключей в А и В **INNER JOIN**
Select <...> From A INNER JOIN B on A.Key = B.Key;

3. Соединение всего из А и совпавшее из В **LEFT JOIN**
Select <...> From A LEFT JOIN B on A.Key = B.Key;

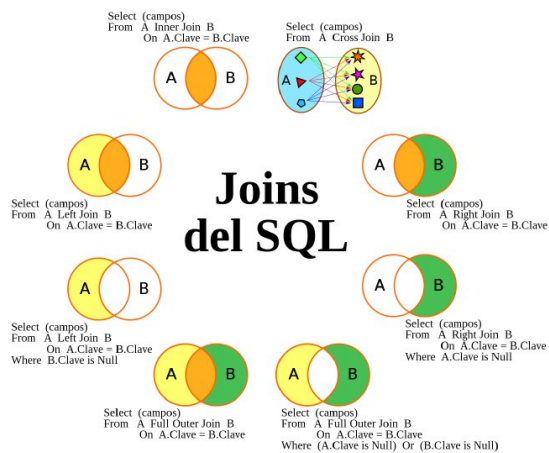
4. Соединение всего из А и чего нет в В **LEFT JOIN (с NULL)**
Select <...> From A LEFT JOIN B on A.Key = B.Key where B.Key is NULL;

5. Соединение всего из В и совпавшее из А **RIGHT JOIN**
Select <...> From A RIGHT JOIN B on A.Key = B.Key;

6. Соединение всего из В и совпавшее из А **RIGHT JOIN (с NULL)**
Select <...> From A RIGHT JOIN B on A.Key = B.Key where B.Key is NULL;

7. Соединение всего из А и всего из В **FULL OUTER JOIN**
Select <...> From A FULL OUTER JOIN B on A.Key = B.Key;

8. Соединение Чего нет в А и Чего нет в В **FULL OUTER JOIN (с NULL)**
Select <...> From A FULL OUTER JOIN B on A.Key = B.Key where A.Key is NULL OR B.Key is NULL;

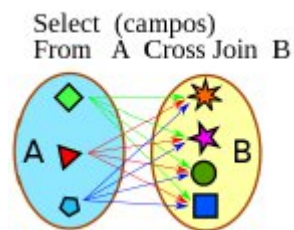


NATURAL JOIN – имена для соединения выбирает СУБД

UNION, INTERSECT, EXCEPT – соединение в высоту

JOIN – соединение в ширину

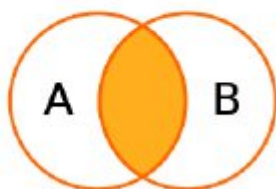
39. Соединение Cross Join. Пример.



```
SELECT admin.id_admin, weekday.id_weekday FROM Admin
CROSS JOIN Weekday;
LEFT JOIN Visit ON Admin.id_admin = Visit.id_admin AND Weekday.id_weekday =
visit.id_weekday
```

40. Соединение Inner Join. Пример.

```
Select (campos)
From A Inner Join B
On A.Clave = B.Clave
```



```
INNER JOIN Pass ON visitor.id_visitor = pass.id_visitor
INNER JOIN Type_of_pass ON pass.id_type_of_pass = type_of_pass.id_type_of_pass
```

41. Соединение Union. Пример.

Получить названия деталей, которые либо имеют цену более 100, либо поставляются поставщиками из Челябинска.

```
SELECT P.Name
FROM P
WHERE P.Price>100
UNION
SELECT P.Name
FROM S, P, SP
WHERE SP.PID=P.PID and SP.SID=S.SID and S.City='Челябинск';
```

Результат 1

Name
Болт
Болт
Шуруп

UNION

Результат 2

Name
Шуруп
Гайка
Гайка



Результат

Name
Болт
Шуруп
Гайка

Результатом объединения будет множество, состоящее из всех строк. Исключаются дубликаты.

Два исходных отношения иметь одинаковый набор имен столбцов.

PID	Name	Price
P1	Болт	200
P2	Гайка	10
P3	Болт	150
P4	Уголок	15
P5	Шуруп	130

SID	PID	Qty
S1	P1	10
S2	P1	40
S3	P5	20
S7	P2	100
S8	P2	300
S5	P4	500

SID	City
S1	Москва
S2	Урюпинск
S3	Челябинск
S5	Одесса
S7	Челябинск
S8	Челябинск

UNION соединение в высоту!

Name
Болт
Болт
Шуруп

UNION ALL

Name
Шуруп
Гайка
Гайка



Результат

Name
Болт
Болт
Шуруп
Шуруп
Гайка
Гайка

Оператор UNION ALL допускает выборку повторяющихся значений.

42. Соединение Intersect. Пример.

Получить названия деталей, которые имеют цену более 100, и поставляются поставщиками из Челябинска.

```
SELECT P.Name
FROM P
WHERE P.Price>100
INTERSECT
SELECT P.Name
FROM S, P, SP
WHERE SP.PID=P.PID and SP.SID=S.SID and S.City='Челябинск';
```

Результат 1

Name
Болт
Болт
Шуруп

INTERSECT

Результат 2

Name
Шуруп
Гайка
Гайка



Результат

Name
Шуруп

Результатом пересечения будет множество, состоящее из строк, присутствующих в обоих запросах.

Исключаются дубликаты.

Два исходных отношения иметь одинаковый набор имен столбцов.

<https://dev.mysql.com/doc/refman/8.0/en/set-operations.html>

PID	Name	Price
P1	Болт	200
P2	Гайка	10
P3	Болт	150
P4	Уголок	15
P5	Шуруп	130

SID	PID	Qty
S1	P1	10
S2	P1	40
S3	P5	20
S7	P2	100
S8	P2	300
S5	P4	500

SID	City
S1	Москва
S2	Урюпинск
S3	Челябинск
S5	Одесса
S7	Челябинск
S8	Челябинск

INTERSECT соединение в высоту!

Name
Болт
Болт
Шуруп

INTERSECT ALL

Name
Шуруп
Гайка
Гайка



Результат

Name
Шуруп

Оператор INTERSECT ALL допускает выборку повторяющихся значений. Правило выборки min(n1,n2)
n1 – число повторений в первом запросе,
N2 – число повторений во втором запросе

43. Соединение Except. Пример.

Получить названия деталей, которые либо имеют цену более 100, и поставляются поставщиками не из Челябинска.

```
SELECT P.Name
FROM P
WHERE P.Price>100
EXCEPT
SELECT P.Name
FROM S, P, SP
WHERE SP.PID=P.PID and SP.SID=S.SID and S.City='Челябинск';
```

Результат 1

Name
Болт
Болт
Шуруп

EXCEPT

Результат 2

Name
Шуруп
Гайка
Гайка



Результат

Name
Болт

Результатом исключения будет множество, состоящее из строк, первого запроса, отсутствующих во втором.

Исключаются дубликаты.

Два исходных отношения иметь одинаковый набор имен столбцов.

PID	Name	Price
P1	Болт	200
P2	Гайка	10
P3	Болт	150
P4	Уголок	15
P5	Шуруп	130

SID	PID	Qty
S1	P1	10
S2	P1	40
S3	P5	20
S7	P2	100
S8	P2	300
S5	P4	500

SID	City
S1	Москва
S2	Урюпинск
S3	Челябинск
S5	Одесса
S7	Челябинск
S8	Челябинск

EXCEPT соединение в высоту!

Name
Болт
Болт
Шуруп

EXCEPT ALL

Name
Шуруп
Гайка
Гайка



Результат

Name
Болт
Болт

Оператор EXCEPT ALL допускает выборку повторяющихся значений.

44. Время жизни таблиц в СУБД. Подзапросы. Места расположения подзапросов.

Подзапросы

Подзапрос – это запрос SELECT, расположенный внутри другого запроса.

По возвращаемому результату:

- скалярные, возвращающие единственное значение;
- векторные, возвращающие от 0 до нескольких элементов;
- табличные, возвращающие таблицу.

Включаются с командами:

- Insert
- Update,
- Delete,
- **Select.**

Содержатся в частях запроса:

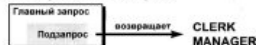
- Select
- from,
- Where,
- having.

Типы подзапросов

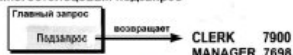
• Однострочный подзапрос



• Многострочный подзапрос

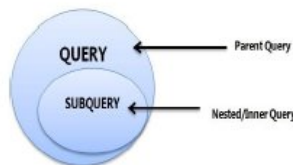


• Многостолбцовый подзапрос



По зависимости от основного запроса:

- некоррелированные – не содержат ссылки на запрос верхнего уровня, вычисляются один раз;
- коррелированные – содержат условия, зависящие от значений полей в основном запросе, вычисляются для каждой строки запроса верхнего уровня.



Правила построения подзапросов:

- Подзапрос заключается в скобки
- Подзапрос стоит справа от оператора сравнения
- Подзапрос не должен содержать ORDER BY

Для запроса **Select**:

WHERE ограничения, чаще всего в IN,
FROM – формирование новых таблиц,
HAVING – аналогично Where,
SELECT – выбор единственного значения.

Таблицы:

- временные (Существуют только во время сеанса (сессии) пользователя или транзакции).
- Постоянные (пока не drop)
- Виртуальные View (Не хранят данные, а являются запросами, которые материализуются при обращении)

Расположение подзапросов в командах:

В команде **INSERT**:

- вместо VALUES:

insert into <tbl> select <atr_name> from <tbl>;

В команде **DELETE**:

- в части WHERE :

delete from <tbl> where <field> IN (select <field> from <tbl> where <cond>);

В команде **UPDATE**:

- в части WHERE для вычисления условий

update <tbl> set <field> = <value> where <field> IN (select <field> from <tbl> where <cond>);

- вместо SET:

update <tbl> set <field> = (select <field> from <tbl> where <cond>);

Расположение подзапросов в команде **SELECT**:

В части SELECT:

Select (select <att> from tbl where <cond>) from <tbl> where <cond>

Легко получить ошибку
этапа исполнения

В части WHERE:

Select <att> from <tbl> where <field> > (select <field> from <tbl> where <cond>);

В части FROM:

Select <att> from select <field> from <tbl> where <cond>) as <name> where <cond>;

45. Подзапросы в части запроса From. Пример.

В части **FROM**: Select <att> from select <field> from <tbl> where <cond>) as <name> where <cond>;

Получить имена поставщиков, поставляющих деталь P2.

select distinct S.Name
from S,

(select S.SID as SID
from S, SP
where SP.SID=S.SID and SP.PID='P2') as SIDs_P2

where S.SID=SIDs_P2.SID;

S		SP		
SID	Name	SID	PID	Qty
S1	Саспенс +	S1	P5	10
S2	ИТ-монстр	S3	P2	20
S3	Разноимпорт	S2	P2	30
S5	МакроХард	S7	P1	40
S7	Овощторг	S5	P4	50
S8	Разноимпорт	S8	P2	30

(select S.SID as SID
from S, SP
where SP.SID=S.SID and
SP.PID='P2')

SIDs_P2

SID
S2
S3
S8

select distinct S.Name
from S,
SIDs_P2
where S.SID=SIDs_P2.SID;

Результат

Name
ИТ-монстр
Разноимпорт

Запрос 4

46. Подзапросы в части запроса Select. Пример.

В части HAVING:

Список отделов, в которых средняя зарплата ниже, чем средняя по предприятию:

```
select depno, avg(salary) sal
  from staff
 group by depno
 having avg(salary) < (select avg(salary)
                      from staff);
```

select depno, avg(salary) sal
from staff
group by depno
having salary < (select avg(salary) from staff);
Работать не будет!
После группировки нельзя обращаться к значениям!

В части SELECT: Select (select <att> from tbl where <cond>) from <tbl> where <cond>

Список сотрудников с указанием количества проектов, в которых они участвуют:

```
select depno, name, count(*)
  from job as j, staff as e
 where j.id = e.id
 group by depno, name
union all
select depno, name, 0
  from staff
 where id not in (select id from job);
```

Te, у кого есть проекты
Te, у кого нет проектов
Работает медленно
Возможны ошибки этапа исполнения
Не во всех СУБД есть
count(*) выполняется для каждой строки и вернет 0
если проектов нет

47. Подзапросы в части запроса Where. Пример.

В части WHERE: Select <att> from <tbl> where <field> > (select <field> from <tbl> where <cond>);

1. Вывести список сотрудников, у которых зарплата выше, чем средняя по предприятию:

```
select id, name, depno, post, salary
  from staff
 where salary > (select avg(salary) from staff);
```

id	name	depno	post	salary
1	1 FOMIN VLADIMIR IVANOVICH	1	Head of Department	62500
2	7 BREZHNEV YURIJ FEDOROVICH	1	Deputy Head of Depart..	56000
3	55 ZYRYANOVA IRINA ALEKSAN..	2	Deputy Director	63000
4	100 VOLIS IOSIF IVANOVICH	6	engineer	54450

2. Вывести список сотрудников, у которых зарплата выше, чем средняя по каждому отделу предприятия

```
select id, name, depno, post, salary
  from staff
 where salary > ALL
      (select avg(salary)
       from staff
       group by depno);
```

id	name	depno	post	salary
1	40 SVININ IVAN MIXAJLOVICH	2	Deputy Director	82500
2	44 SEDOV VALERIJ IVANOVICH	2	Director	93500
3	60 MOZGALEV ARKADIJ MIXAJ..	5	Head of Department	68750
4	88 GRIGOR' EVA MARIYA FEDO..	6	Head of Department	68750
5	108 FARAFONOV ALEKSANDR IV..	8	Head of Department	68750
6	145 ASHUROV MIXAIL DAVY' DO..	7	Head of Department	68750

3. Выдать список сотрудников, имеющих детей:

а) с помощью операции соединения таблиц:
SELECT distinct e.*
FROM staff as e, children as c
WHERE e.id=c.id;

б) с помощью некоррелированного векторного подзапроса:
SELECT *
FROM staff
WHERE id IN (SELECT id FROM children);

в) с помощью коррелированного табличного подзапроса:
SELECT *
FROM staff as e
WHERE EXISTS
(SELECT * FROM children as c
WHERE e.id=c.id);

Вывести список сотрудников, у которых зарплата выше, чем средняя в отделе, в котором работает данный сотрудник:

```
select * from staff as e
      where salary > (select avg(salary) from staff as m
                     where m.depno = e.depno);
```

Коррелированный подзапрос.
Вычисляется долго!

Условие соединения в where:

```
select e.* from staff as e,
      (select depno, avg(salary) sal
       from staff
       group by depno) as m
 where m.depno = e.depno and salary > sal;
```

Подзапрос вычисляется 1 раз

Запросы эквивалентны

Условие соединения в on:

```
select e.* from staff as e,
      (select depno, avg(salary) sal
       from staff
       group by depno) as m on m.depno = e.depno
 where salary > sal;
```

Запрос 6

48. Определение представления. Назначение. Операции над представлениями. Пример.

Представление — объект базы данных, являющийся результатом выполнения запроса к базе данных, определённого с помощью оператора **SELECT**, в момент обращения к представлению.

Представления иногда называют «виртуальными таблицами». Это связано с тем, что представление для пользователя выглядит как таблица, но фактически не хранит данных, а извлекает их из других таблиц в момент обращения.

Если данные в основной таблице меняются, пользователь получает актуальные данные при обращении к представлению, использующему эту таблицу. При этом представления не кэшируют результаты выборки из таблицы в процессе работы.

Представление (View) хранимый запрос или результат запроса Select.

Особенности:

- является виртуальной таблицей;
- может быть использовано в других запросах;
- доступно, в основном, для чтения.

Назначение:

- хранение сложных запросов;
- представление данных в виде, удобном пользователю;
- сокрытие конфиденциальной информации;
- разграничение доступа к данным.

1. Создание представления:

```
CREATE
[OR REPLACE]
[ALGORITHM = {UNDEFINED | MERGE | TEMPTABLE}]
[DEFINER = user]
[SQL SECURITY { DEFINER | INVOKER }]
VIEW view_name [(column_list)]
AS select_statement
[WITH [CASCADED | LOCAL] CHECK OPTION]
```

```
CREATE [OR REPLACE] VIEW <name>
[(<field>)] AS <Select> [WITH CHECK OPTION];
```

Ограничения представлений. **Нельзя:**

- «повесить» триггер;
- использовать временные таблицы в качестве источника;
- сделать временное представление;
- включить подзапрос во From.

2. Изменение представления:

```
ALTER
[ALGORITHM = {UNDEFINED | MERGE | TEMPTABLE}]
[DEFINER = user]
[SQL SECURITY { DEFINER | INVOKER }]
VIEW view_name [(column_list)]
AS select_statement
[WITH [CASCADED | LOCAL] CHECK OPTION]
```

```
ALTER VIEW <name> [(<field>)] AS
<Select> [WITH CHECK OPTION];
```

Особенности реализации:

- MERGE — ссылки на исходные таблицы - «виртуальное»;
- TEMPTABLE — создание новой таблицы - «материализованное»

Требования к обновляемости:

- связь «1 к 1» с исходными таблицами;
- отсутствие вычисляемых полей.

3. Удаление представления:

```
DROP VIEW [IF EXISTS]
view_name [, view_name] ...
[RESTRICT | CASCADE]
```

```
DROP VIEW <name>
```

2. Создать представление "Сотрудники 3-го отдела" (для предоставления доступа на чтение к данным о сотрудниках 3-го отдела начальнику этого отдела):

```
CREATE OR REPLACE VIEW staff3
AS SELECT *
FROM staff
WHERE depno = 3
WITH CHECK OPTION;
```

```
SELECT * FROM staff3;
```

Создали представление

```
UPDATE staff3
SET depno = 5
WHERE id = 86;
```

Изменили строку представления
Но запрос не будет выполнен, так
как изменение противоречит
правилу представления



```
SELECT * FROM staff;
```

Не увидим результат в таблице и в
представлении

Представления могут основываться как на таблицах, так и на других представлениях, т.е. могут быть вложенными (до 32 уровней вложенности).

49. Определение триггера. Назначение, Способ функционирования. Операции над триггерами.

Начало транзакции - при появлении первого оператора

Завершение транзакции:

- поступила команда commit - фиксация.
- поступила команда rollback - откат.
- успешно завершена программа (exit, quit) - фиксация.
- программа завершена аварийно (abort) - откат.

Фиксация транзакции:

1.Изменения, внесённые транзакцией, помечаются как постоянные.

2. Удаляются точки сохранения транзакции.

3.Освобождаются объекты, заблокированные транзакцией.

4. В журнале транзакций транзакция помечается как завершённая.

Механизм реализации транзакций в СУБД

– **журнал транзакций** содержит:

- номер транзакции;
- состояние транзакции: завершена фиксацией или откатом, не завершена, находится в состоянии ожидания;
- точки сохранения (явные и неявные);
- команды, составляющие транзакцию.

Откат транзакции:

1. Считываются старые значения и возвращаются в БД.

Выполняется медленнее фиксации

Идеальное выполнение транзакций – они полностью изолированы друг от друга.

Транзакции не влияют друг на друга, если:

- 1. они только читают данные;
- 2. они обращаются к разным данным.

Взаимовлияние транзакций может проявляться в виде:

- 1.потери изменений;
- 2.чернового чтения;
- 3.неповторяемого чтения;
- 4.фантомов.

Пример с COMMIT:

```
set autocommit=0; //отключаем autocommit
Start transaction; (также, можно написать BEGIN; )
...какие-то действий с БД (insert, update,delete...)
commit; //Фиксация действий, запись их в физическую БД
```

Пример с ROLLBACK:

```
set autocommit=0; //отключаем autocommit
Start transaction;
...какие-то действия с БД (insert, update,delete...)
rollback; // отменяем серию действий, не производим запись в физическую БД
```

В MySQL не существует механизма вложенных транзакций. Одно соединение с БД — одна транзакция. Новая транзакция в пределах одного соединения может начаться только после завершения предыдущей.

Для некоторых операторов нельзя выполнить откат с помощью ROLLBACK. Это операторы языка определения данных (Data Definition Language — DDL). Сюда входят запросы CREATE, ALTER, DROP, TRUNCATE, COMMENT, RENAME.

52. Блокировки на чтение и на запись. Взаимодействие блокировок. Явные и неявные блокировки.

Блокировка на чтение – нельзя читать, пока не закончена другая транзакция (потеря изменений, грязное чтение)

Блокировка на запись – нельзя записывать, пока не закончена другая транзакция

Блокировка – это временное ограничение доступа к данным, участвующим в транзакции, со стороны других транзакций.

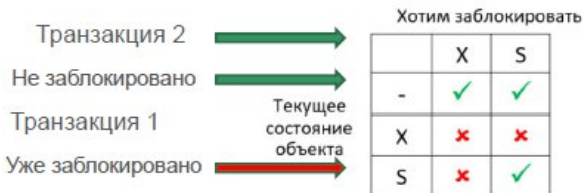
СУБД обеспечивает блокировки двух типов:

- S-блокировка (Shared) - разделяемая блокировка или блокировка допускающая чтение. Обеспечивает совместный доступ на чтение объекта.
- X-блокировка (Xclusive) - эксклюзивная блокировка или блокировка для записи. Не допускает совместный доступа к объекту даже на чтение.

Блокировки одних и тех же объектов:

по чтению несколькими транзакциями совместимы -
нескольким транзакциям допускается одновременно читать
один и тот же объект.

по записи разными транзакциями не совместимы -
никакой транзакции нельзя изменять объект, изменяемый
некоторой транзакцией.



Блокировка объекта одной транзакцией по чтению не совместима с блокировкой другой транзакцией того же объекта по записи:

- никакая транзакция не может изменять объект, читаемый некоторой транзакцией (кроме самой этой транзакции)
- никакой транзакции нельзя читать объект, изменяемый некоторой транзакцией (кроме самой этой транзакции)

Режимы блокировки:

- неявные блокировки при выполнении транзакций;
- явные блокировки - сделанные из кода.

Цель явных блокировок:

- загрузка данных из файла в таблицу;
- проведение операций, требующих явной блокировки на заданном уровне изоляции.

Синтаксис выбор таблицы для блокировки:

```
LOCK TABLES
  tbl_name [[AS] alias] lock_type
  [, tbl_name [[AS] alias] lock_type] ...

lock_type: {
  READ [LOCAL]
  | [LOW_PRIORITY] WRITE
}

UNLOCK TABLES
```

```
LOCK TABLES tbl_name [AS alias]
{READ | WRITE} [, tbl_name {READ | WRITE} ...]
...
UNLOCK TABLES
```

Потеря изменений (Lost updates)



O₀ – единственная копия кортежа

T₁ – изменяет объект O₀ на O₁

**СУБД не допускает
взаимовлияния транзакций
с потерей изменений**

T₂ – изменяет объект O₀ на O₂

T₂ – откатывает объект O₂ на O₀

T₁ – читает объект O ждет O₁ получает O₀

«Грязное» чтение (dirty read)



O₀ – единственная копия кортежа

T₁ – изменяет объект O₀ на O₁

T₂ – Читает объект O и видит O₁

T₁ – откатывает объект O₁ на O₀

T₂ – продолжает считать что O равно O₁ а оно O₀

Неповторяющееся чтение (non-repeatable read)



O₀ – единственная копия кортежа

T₁ – Читает объект O и получает O₀

T₂ – меняет объект O и пишет O₁

T₂ – сохраняет изменения транзакции – в базе O₁

T₂ – читает O и получает O₁ а оно было O₀

Транзакция T₁ при повторном чтении получает иной результат

Чтение фантомов (phantom reads)



Транзакция T₁ при повторном чтении получает иное число записей

Tab – единственная копия таблицы

T₁ – Читает Tab и получает n записей

T₂ – добавляет в Tab 1 новую запись

T₂ – сохраняет изменения транзакции – в базе

T₁ – Читает Tab и получает n+1 записей, было n записей

53. Уровни изоляции и исключение проблем совместного доступа.

В СУБД используются методы регулирования совместного выполнения транзакций. Проблема скорости выполнения!

$S = \{T_1, T_2, \dots, T_n\}$ - множество одновременно выполняемых транзакций

Требуется построить сериальный план при котором последовательно-параллельное выполнение транзакций будет эквивалентно последовательному.

Конфликты между транзакциями T_1 и T_2 :

Вид конфликта	Описание	Проблема
WW (конфликт Write/Write)	транзакция T2 пытается изменить объект, изменённый не закончившейся транзакцией T1	потерянных изменений
RW (конфликт Read/Write)	транзакция T2 пытается изменить объект, прочитанный не закончившейся транзакцией T1	неповторяющиеся чтения
WR (конфликт Write/Read)	транзакция T2 пытается читать объект, изменённый не закончившейся транзакцией T1	«грязного» чтения

Конфликт Read/Read – отсутствует. Только читающие транзакции не конфликтуют.

Синтаксис выбора уровня изоляции:

```
SET [GLOBAL | SESSION] TRANSACTION
transaction_characteristic [, transaction_characteristic] ...

transaction_characteristic: {
  ISOLATION LEVEL level
  | access_mode
}

level: {
  REPEATABLE READ
  | READ COMMITTED
  | READ UNCOMMITTED
  | SERIALIZABLE
}

access_mode: {
  READ WRITE
  | READ ONLY
}
```

Уровни изоляции в ISO/ANSII:

Уровень изоляции	Грязное чтение	Неповторяемое чтение	Фантомы
Read Uncommitted – чтение незавершённых транзакций	да	да	да
Read Committed – чтение завершённых транзакций	нет	да	да
Repeatable Read – повторяемое чтение	нет	нет	да
Serializable – последовательное чтение	нет	нет	нет

Не во всех СУБД реализовано так!

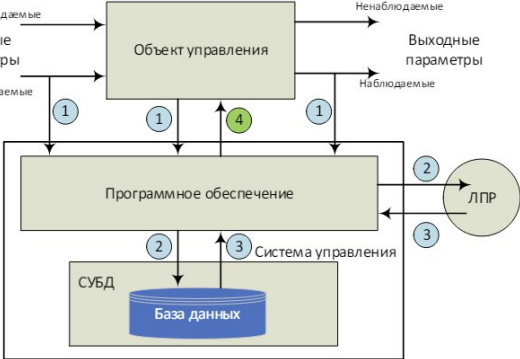
54. Место БД в системе управления. Схема. Цель управления.

Схема – описывает логическую структуру базы данных
БД – физический уровень (представление схемы в виде файлов файловой системы)



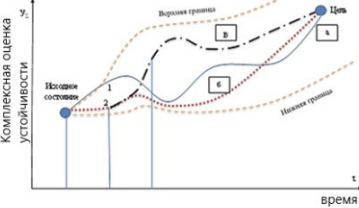
ИНФОРМАЦИОННАЯ СИСТЕМА

Цель – обеспечение устойчивого управления для достижения результата



ЛПР – лицо принимающее решение

Число функций > 0 автоматизированное
Число функций = 0 автоматическое

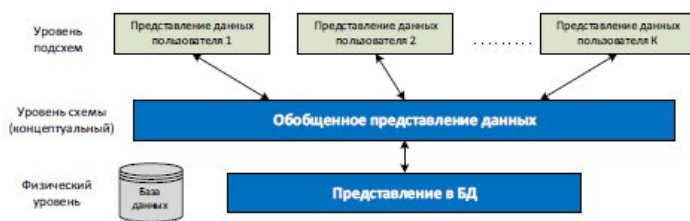


Число параметров велико, критерии оптимальности противоречивы!

2

55. Уровни абстракции в архитектуре СУБД. Уровень подсхем, концептуальный и физический уровни.

Подсхема – представление БД конечным пользователем (внешний уровень)
Концептуальный уровень – логическая структура всей бд
Физический уровень – хранение данных на диске



Уровень схемы: формируется системным администратором

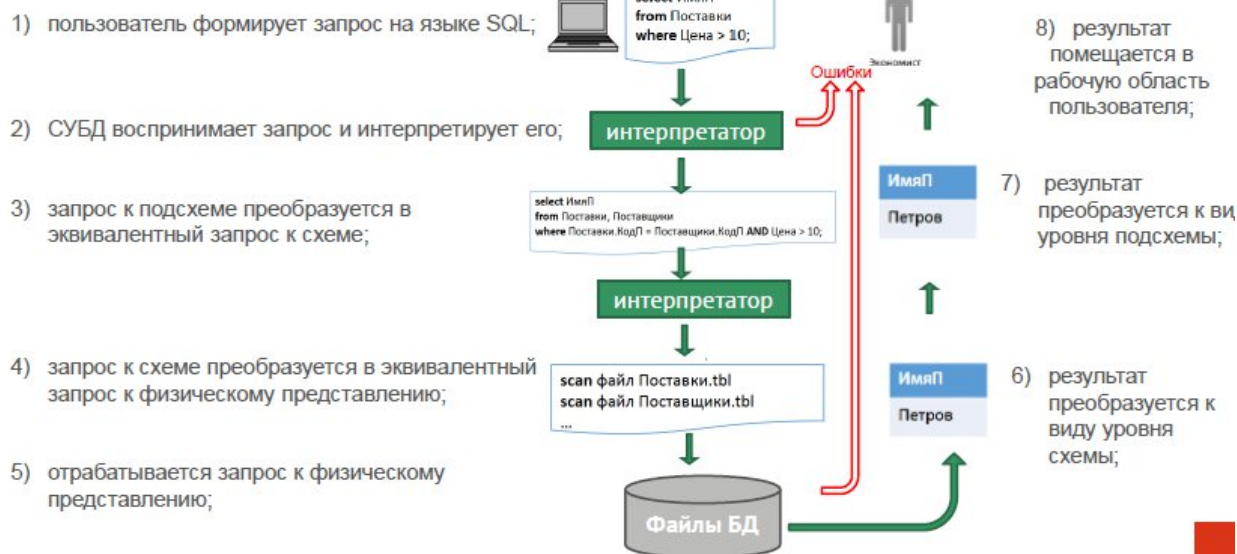
Уровень подсхем: определяется правами доступа

Физический уровень реализуется средствами конкретной СУБД

Физический уровень – представление схемы в виде файлов файловой системы

на C:\ProgramData\MS\SQL\Server\8.0\Data\... (из книги)

56. Цикл выполнения запроса. Алгоритм выполнения запроса на выборку.



Общий алгоритм выполнения операции SELECT

1. Статический синтаксический анализ конструкций (Select, from, where).
2. Динамический анализ принадлежности атрибутов таблиц (уникальность, правильность имени).
3. Проверка прав доступа к таблицам и атрибутам.
4. Построение плана запроса (построение последовательностей соединения таблиц).
5. Проверка для каждой строки каждой таблицы условия отбора (where).
6. Извлечение из полученных записей тех полей, которые заданы в списке вывода (select), и формирование ответа.
7. Выполнение соединения таблиц (вложенных запросов).
8. Сортировка для получения группировки.
9. Группировка полученных в результате отбора записей (group by).
10. Вычисление для этих групп значений агрегирующих функций.
11. Если присутствуют конструкции UNION, INTERSECT, MINUS. Повторить п.п.1-10 для каждого запроса.
12. Соединить по правилу п.11. результаты запросов.
13. Выбор тех групп, которые удовлетворяют условию отбора групп (having).
14. Сортировка полученных записей в указанном порядке (order by).

57. Системный каталог. Назначение. Хранимые данные.

В системном каталоге хранятся:

- имена, типы и размеры элементов данных;
- имена связей;
- накладываемые на данные ограничения целостности;
- имена пользователей, которым предоставлено право доступа к данным;
- внешняя, концептуальная и внутренняя схемы и отображения между ними;
- статистические данные, например частота транзакций и счетчики обращений к объектам базы данных.

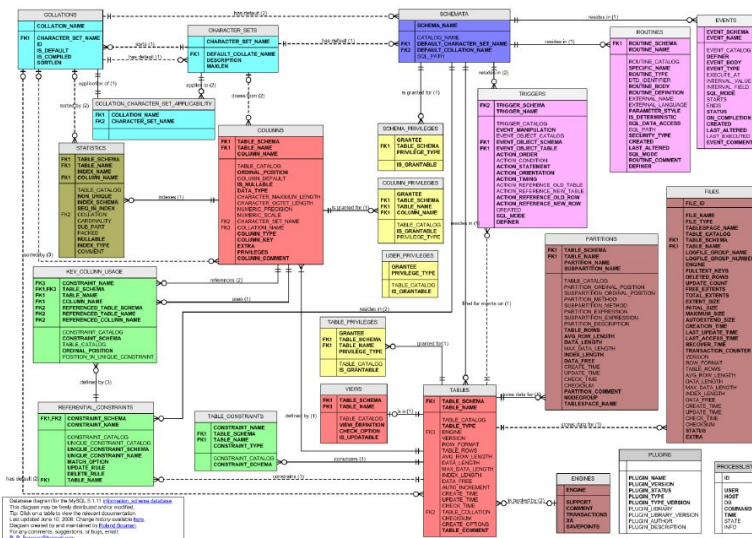
Преимущества наличия ССД

- информация о данных может быть централизованно собрана и сохранена;
- общий доступ поможет другим пользователям понять предназначение данных;
- интегрированы права на данные и данные;
- благодаря централизованному хранению исключается избыточность и противоречивость данных;
- внесенные в базу данных изменения могут быть заprotoколированы;
- последствия любых изменений могут быть определены еще до их внесения;
- меры обеспечения безопасности могут быть дополнительно усилены;
- появляются новые возможности организации поддержки целостности данных;
- может выполняться аудит данных.

Системный каталог – хранилище метаданных базы данных

Системный каталог реализован в системной части СУБД доступ осуществляется INFORMATION_SCHEMA

58. Пример системного каталога Information_schema.



INFORMATION_SCHEMA –
отображение системного каталога для пользователя

- темно синий – схемы данных;
- светло-синий - наборы символов и последовательности сортировок;
- красный основные таблицы;
- темно-зеленый – статистика для построения индексов;
- желтый права пользователей на объекты ;
- зеленый – программные компоненты;
- коричневый – физические характеристики таблиц и процедур.

INFORMATION_SCHEMA –
построена на составных естественных ключах

59. Сетевые архитектуры. Классификация. Типы архитектур. Составные части информационной системы.

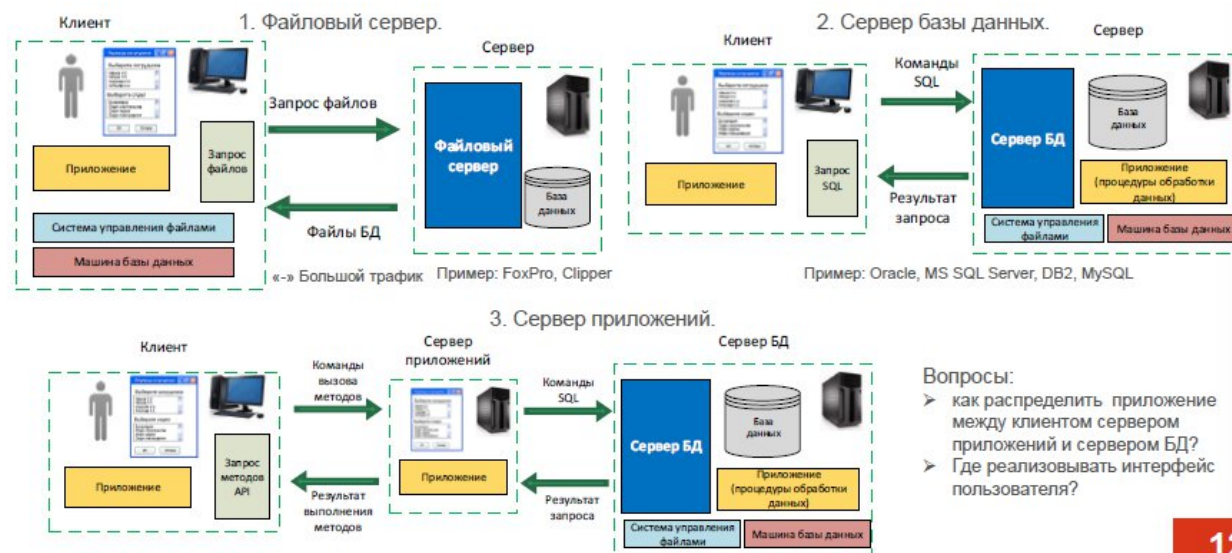
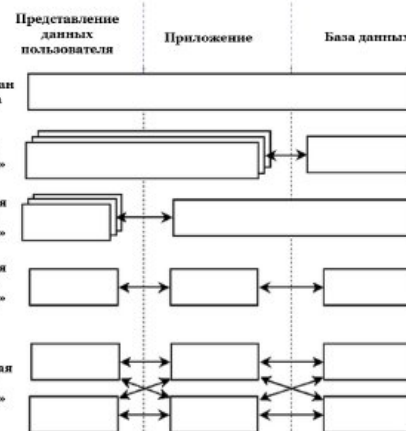
Сетевая архитектура – способ распределения компонентов СУБД между вычислительными узлами сети.

Компоненты системы баз данных:

- пользовательский интерфейс;
- прикладные функции;
- машина базы данных;
- система управления файлами.



Распределение функций между уровнями



Вопросы:

- как распределить приложение между клиентом сервером приложений и сервером БД?
- Где реализовывать интерфейс пользователя?

60. Современная сетевая архитектура Клиент сервер. Схема.

Архитектура клиент-сервер



Толстый клиент:

Большая часть логики приложения и обработки данных происходит на стороне клиентского устройства. Приложение устанавливается локально и требует более мощного оборудования.

Тонкий клиент:

Большая часть логики приложения и обработки данных происходит на сервере, а клиентское устройство лишь отображает интерфейс и обрабатывает ввод пользователя. Тонкий клиент требует меньше ресурсов на клиенте, но требует надежной сети.

61. Понятие сбоя в информационной системе. Типы сбоев. Мягкие и жесткие сбои.

Защита данных – это организационные, программные и технические методы и средства, направленные на удовлетворение ограничений, установленных для типов данных или экземпляров типов данных в системах обработки данных (ГОСТ 20886-85).

Тип доступа:
 > случайный;
 > несанкционированный.

Операции:
 > ознакомление;
 > изменение;
 > удаление.

Источник доступа:
 > пользователи;
 > сторонние лица;
 > аппаратура.

Обеспечение безопасности данных

Цель: предотвращение разрушения или искажения данных в результате программного или аппаратного сбоя.

Задача: восстановить подтвержденные транзакции, исключить не законченные и вернуть БД к последнему непротиворечивому состоянию.

решается на уровне СУБД

Типичные сбои и способы защиты от них:

1. Сбой запроса. СУБД откатывает результаты этого запроса, формирует код ошибки и возвращает управление программе пользователя.
2. Сбой приложения. СУБД откатывает все транзакции приложения, освобождает все ресурсы приложения.
3. Сбой процесса (копии) сервера. Процесс сервера завершается, автоматически происходит откат всех незавершенных транзакций в этом процессе.
4. Сбой корневого процесса сервера. Сервера прекращает выполнение программ, Восстановление требует участия администратора
5. Ошибки пользователя. В зависимости от типа ошибки и от привилегий пользователя устранение требует участия администратора.

Сбои программ — это нарушения нормального режима их функционирования. Проблемы такого рода приводят не только к невозможности работы с программой, но и к риску несанкционированного доступа к данным со стороны посторонних лиц.

Мягкий сбой – утрата оперативной памяти системы

Жесткий сбой – повреждение внешних носителей памяти

62. Резервное копирование Дамп базы данных. Назначение. Место хранения. Периодичность выполнения.

Резервное копирование периодическое сохранение файлов БД на внешнем запоминающем устройстве. Выполняется в непротиворечивом состоянии БД.

По объему:

- полное;
- частичное инкрементное.

По частоте:

- еженедельное;
- ежедневное.

Выполняется:

- администратором системы вручную;
- по расписанию.

Восстановление производится после отказа.

Данные, измененные между созданием последней копии и моментом сбоя будут утеряны.

Создание копии (дампа).

БД будет явно или неявно остановлена!

```
mysqldump [options] db_name [tbl_name ...]
mysqldump [options] --databases db_name ...
mysqldump [options] --all-databases
```

Полезные опции:

- -p<password>;
- -u<user>
- -help.

```
mysqldump [options] > dump.sql
```

Хранить данные на другом носителе!

Восстановление из копии (дампа).

В БД требуется удалить старые данные!

```
mysql db_name < script.sql > output.tab
```

Script sql – файл с резервной копией.

Восстанавливать СУБД следует в монопольном режиме с правами администратора

63. Журнал. Назначение журнала. Метод сохранения данных в журнале. Восстановление после мягкого сбоя.

Для полного восстановления данных используется журнал транзакций – резервная копия в СУБД

Журналирование изменений — функция СУБД, которая сохраняет информацию, необходимую для восстановления базы данных в предыдущее согласованное состояние в случае логических или физических отказов.

Объем данных для журналирования:

- порядковый номер, тип и время изменения;
- идентификатор транзакции;
- объект, подвергшийся изменению;
- предыдущее состояние объекта и новое состояние объекта.

Журнал – последовательная запись операций

Виды восстановления данных:

- *Индивидуальный откат транзакции.* Откат индивидуальной транзакции может быть инициирован либо самой транзакцией путем подачи команды ROLLBACK, либо системой.
- *Мягкий сбой системы (аварийный отказ программного обеспечения).* Мягкий сбой характеризуется утратой оперативной памяти системы.
- *Жесткий сбой системы (аварийный отказ аппаратуры).* Жесткий сбой характеризуется повреждением внешних носителей памяти.

Индивидуальный откат транзакции

- выбирается очередное действие из списка транзакции;
- выполняется противоположная по смыслу операция: вместо INSERT - DELETE, вместо DELETE - INSERT, в UPDATE переворачиваются данные;
- все операции журналируются. На случай мягкого сбоя чтобы выполнить индивидуальный откат;
- при успешном завершении отката в журнал заносится запись о конце транзакции.

Восстановление после мягкого сбоя

Нарушилась целостность страниц данных в ОП, данные на диске нарушены.

Сохранился журнал



Физическая согласованность момент, когда данные на диске и в ОП одинаковы. (сохранение буферов)

T1 - никаких действий производить не требуется. Выполнена полностью.

T2 – выполнить остаток транзакции из журнала и завершить.

T3 – откатить первую часть транзакции.

T4 – полностью выполнить транзакцию.

T5 – ничего не делать. Данных о ней на диске нет.

Прикладные процессы:

T1, T2, T4 получают результат,
T3 и T5 – ошибку выполнения запроса.

64. Разграничение доступа. Операторы создания пользователей, назначения и отмены прав доступа.

Под функцией защиты данных понимается исключение преднамеренного искажения и/или просмотра данных пользователями или посторонними лицами.

Защита данных обеспечивается:

- назначением группам пользователей прав доступа к группам данных в соответствии с правилами;
- организация системы контроля доступа к данным;
- тестирование вновь создаваемых средств защиты данных.

Вид защиты и соответствующее действие	Название привилегии	Применимо к следующим объектам
Просмотр	SELECT	Таблицы, столбцы, подпрограммы, вызываемые из SQL
Вставка	INSERT	Таблицы, столбцы
Модификация	UPDATE	Таблицы, столбцы
Удаление	DELETE	Таблицы
Ссылка	REFERENCES	Таблицы, столбцы
Использование	USAGE	Домены, определенные пользователями типы, наборы символов, порядки сортировки символов, преобразования
Иниципирование	TRIGGER	Таблицы
Выполнение	EXECUTE	Подпрограммы, вызываемые из SQL

Каждый пользователь определяется

- идентификатором (login);
- паролем;
- перечнем разрешённых операций.

Требования при задании пароля:

- длина не менее 6-и символов;
- должен содержать комбинацию букв и цифр или специальных знаков;
- не может содержать пробелы;
- пароли должны часто меняться.

Защита данных обеспечивается:

- созданием пользователей;
- созданием ролей.

Пользователю ограничивается числом:

- запросов,
- изменений,
- соединений в час.

Учетную запись можно:

- заблокировать;
- разблокировать;
- изменить пароль.

Имя пользователя в пределах базы данных должно быть уникальным. Обычно: <имя хоста>@<имя пользователя> 'jeffrey'@'localhost'

Создание записи пользователя

```
CREATE USER [IF NOT EXISTS]
  user [auth_option] [, user [auth_option]] ...
  DEFAULT ROLE role [, role] ...
  [REQUIRE {NONE | tls_option} [(AND) tls_option] ...]
  [WITH resource_option [resource_option] ...]
  [password_option | lock_option] ...
  [COMMENT 'comment_string' | ATTRIBUTE 'json_object']
```

Изменение записи пользователя

```
ALTER USER [IF EXISTS]
  user [auth_option] [, user [auth_option]] ...
  [REQUIRE {NONE | tls_option} [(AND) tls_option] ...]
  [WITH resource_option [resource_option] ...]
  [password_option | lock_option] ...
  [COMMENT 'comment_string' | ATTRIBUTE 'json_object']
```

Удаление записи пользователя

```
DROP USER [IF EXISTS] user [, user] ...
```

Изменение имени записи

```
RENAME USER old_user TO new_user
[, old_user TO new_user] ...
```

Роль – это совокупность привилегий, предоставляемых пользователю и/или другим ролям.

Роль объединяет права и потом назначается пользователю

Установить роль текущему пользователю

Создание роли

Удаление роли

```
CREATE ROLE [IF NOT EXISTS] role [, role] ...
```

```
DROP ROLE [IF EXISTS] role [, role] ...
```

Сценарий:

- создать роль;
- назначить роли права;
- создать запись пользователя;
- Назначить пользователю роль.

```
SET ROLE {
  DEFAULT
| NONE
| ALL
| ALL EXCEPT role [, role] ...
| role [, role] ...
}
```

Права назначаются:

- глобально («*»);
- на базу данных;
- на таблицу;
- на колонку.

При назначении прав:

- запретить передавать права;
- разрешить передавать права.

Удаление прав:

Назначение прав:

```
GRANT
  priv_type [(column_list)]
  [, priv_type [(column_list)]] ...
  ON [object_type] priv_level
  TO user_or_role [, user_or_role] ...
  [WITH GRANT OPTION]
  [AS user
    [WITH ROLE
      DEFAULT
    | NONE
    | ALL
    | ALL EXCEPT role [, role] ...
    | role [, role] ...
    ]
  ]
}
```

```
REVOKE [IF EXISTS]
  priv_type [(column_list)]
  [, priv_type [(column_list)]] ...
  ON [object_type] priv_level
  FROM user_or_role [, user_or_role] ...
  [IGNORE UNKNOWN USER]

REVOKE [IF EXISTS] ALL [PRIVILEGES], GRANT OPTION
  FROM user_or_role [, user_or_role] ...
  [IGNORE UNKNOWN USER]
```

GRANT OPTION FOR – отмена права передачи привилегий
CASCADE – при отмене привилегий у пользователя отменяются все привилегии, переданные другим пользователям;
RESTRICT – запрет отмены привилегий у пользователя если надо менять права других пользователей.

WITH GRANT OPTION – возможность передавать права доступа другим пользователям